Proyecto de big data e inteligencia artificial

Fase 2

StockPulse

CURSO DE ESPECIALIZACIÓN EN

INTELIGENCIA ARTIFICIAL Y BIG DATA

MODALIDAD ONLINE

10/03/2026

Jimmy Chen
Joaquín Fernández
Sergio Romero
Deninson Tapia

Repositorio: https://github.com/Joaquin-AI-DEV/Proyecto_IA_BigData_Grupo5

1. Estado actual del proyecto **3**
2. Guía de ejecución local **3**
2.1 Requisitos previos **3**
2.2 Paso 1 — Clonar el repositorio **4**
2.3 Paso 2 — Crear y activar el entorno virtual **4**
2.4 Paso 3 — Instalar dependencias **4**
2.5 Paso 4 — Configurar variables de entorno **4**
2.6 Paso 6 — Ejecución del proyecto, Cargar CSV y Inicio de sesión **4**
2.7 Paso 7 (opcional) — Verificar sin conexión a BD **5**
3. Evidencias funcionales **5**
4. Arquitectura del sistema implementada **10**
4.1 Cambio tecnológico: MySQL → Supabase (PostgreSQL) **10**
4.2 Capas del sistema **10**
4.3 Flujo de datos **11**
4.4 Separación de responsabilidades **11**
5. Índice del código — Estructura del proyecto **12**
6. Código completo del proyecto **13**
7. Breve explicación por módulo **74**
8. Limitaciones actuales **76**
8.1 Bugs detectados **77**
8.2 Atajos implementados **77**
8.3 Partes incompletas 77

1. Estado actual del proyecto

Actualmente el sistema cuenta con las siguientes funcionalidades:

- Pipeline de datos
  - Ingesta de datos
  - Limpieza y transformación
  - Carga en la base de datos
  - Ejecución orquestada del pipeline

- Backend funcional
  - Estructura modular organizada
  - Conexión a base de datos implementada
  - Definición de modelos

- Gestión del entorno
  - Configuración mediante .env
  - Uso de entorno virtual venv
  - Dependencias definidas en requirements.txt

Mejoras a implementar:

- Uso de pantallas de carga tanto al ejecutar el pipeline y el modelo como al borrar la Base de datos.
- Cierre de sesión más eficiente.
- Mejora de visualización de gráfica circular.
- Creación y almacenamiento de usuarios.
- División del frontend en varios archivos para mejorar la accesibilidad.
- Mejora de la precisión del modelo.
- Mejora del pipeline a la hora de limpieza de datos
- Implementación de un botón para alternar entre modo claro y oscuro para accesibilidad.
- Mejor visualización de gráficas.

2. Guía de ejecución local

Esta guía permite levantar el pipeline de datos de StockPulse desde cero en cualquier máquina con Python 3.10+. No se necesita instalar ningún servidor de base de datos local; la BD está en Supabase.

2.1 Requisitos previos

- Python 3.10 o 3.11
- pip (gestor de paquetes de Python)
- Git
- Acceso a internet (para Supabase)
- El archivo del dataset: Online Retail Data Set.xlsx (descargable desde Kaggle: https://www.kaggle.com/datasets/bekkarmerwan/retail-sales-dataset-sample-transactions)

2.2 Paso 1 — Clonar el repositorio

```
git clone https://github.com/Joaquin-AI-DEV/Proyecto_IA_BigData_Grupo5.git
cd Proyecto_IA_BigData_Grupo5
```

2.3 Paso 2 — Crear y activar el entorno virtual

```
# Crear entorno virtual
python -m venv venv

# Activar en Windows
venv\Scripts\activate

# Activar en Linux / Mac
source venv/bin/activate
```

2.4 Paso 3 — Instalar dependencias

```
pip install -r environment/requirements.txt
```

2.5 Paso 4 — Configurar variables de entorno

Copia el archivo de ejemplo y rellena las credenciales de Supabase:

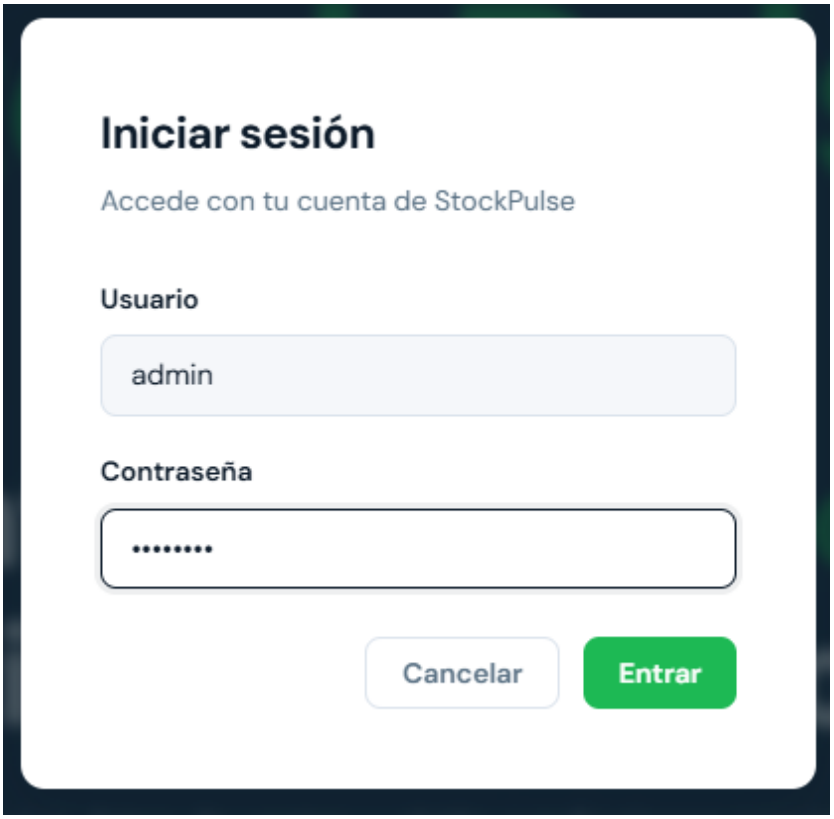
```
# Linux / Mac
cp environment/.env.example environment/.env

# Windows
copy environment\.env.example environment\.env
```

El archivo .env debe contener:

```
DB_HOST=aws-0-eu-west-1.pooler.supabase.com
DB_PORT=6543
DB_NAME=postgres
DB_USER=postgres.nbamabcfxtcltbzxuncg
DB_PASSWORD=123Grupo5321
```

2.6 Paso 6 — Ejecución del proyecto, Cargar CSV y Inicio de sesión

Ejecutar este comando: uvicorn main:app --reload --port 8000
Debes tener la máquina virtual encendida y las dependencias instaladas y encontrarte en la carpeta src/backend.



Iniciar sesión con las credenciales:
Usuario: admin
Contraseña: admin123

Pulsar el botón Cargar nuevo CSV, esto ejecuta el proceso de pipeline convirtiendo el .xslx a csv y ejecuta el modelo.

El pipeline muestra el progreso por consola y al finalizar correctamente imprime:

```
============================================================
   Pipeline completado con éxito.
============================================================
```

2.7 Paso 7 (opcional) — Verificar sin conexión a BD

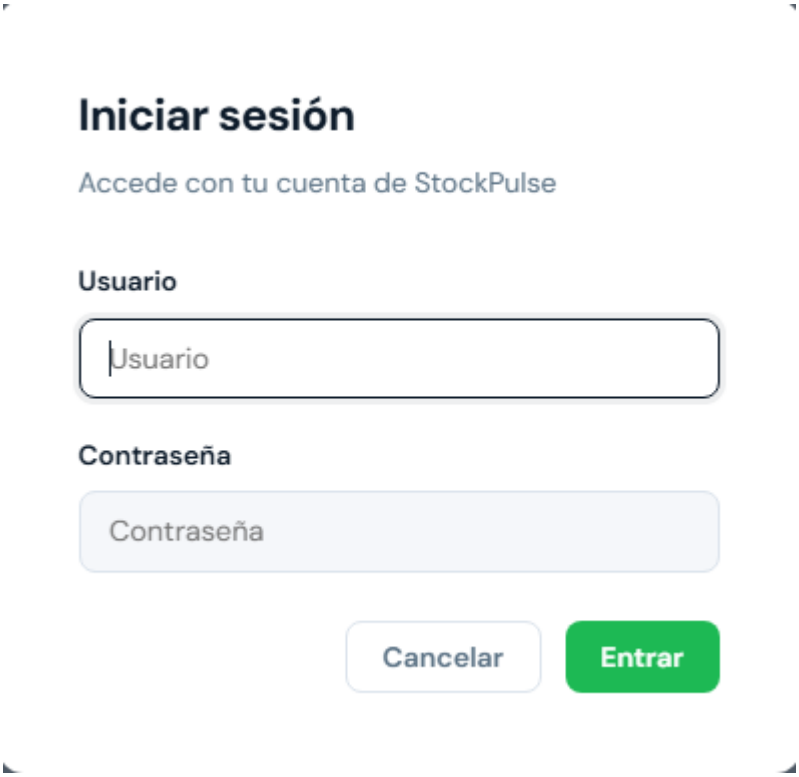
Si no hay acceso a Supabase en ese momento, el test local verifica las capas de ingesta y limpieza de forma independiente:

```
python test_pipeline_local.py
```

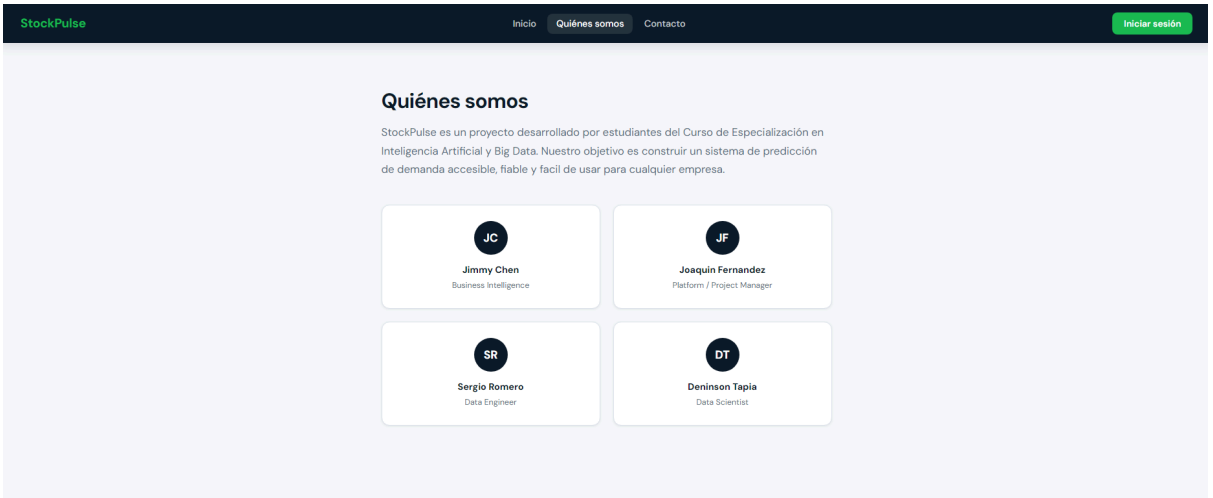
3. Evidencias funcionales

Vista principal de StockPulse. La navbar muestra el nombre del proyecto y los enlaces de navegación. El botón de carga redirige al login si no hay sesión activa.

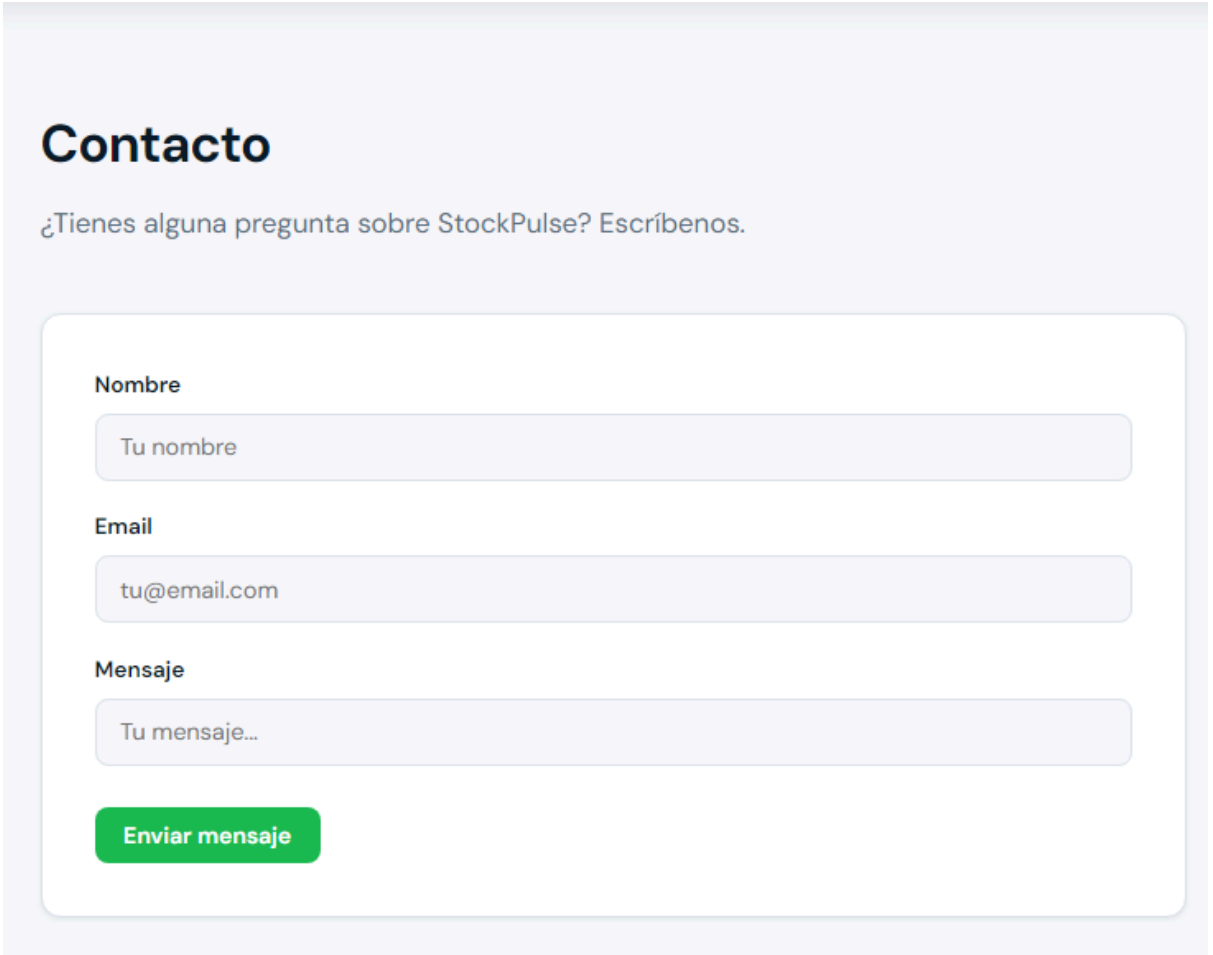
Modal de autenticación. La sesión se gestiona exclusivamente en memoria, sin persistencia en disco ni opción de recordar credenciales.



Página de presentación del equipo. Muestra a los cuatro integrantes con su rol en el proyecto.



Página de contacto. Se piden nombre, email y el mensaje que se quiere transmitir.
.



Ejecución del pipeline sobre el dataset real de 541.909 filas. Las capas de ingesta, limpieza y exportación a CSV funcionan correctamente. La inserción en Supabase queda pendiente de resolución de credenciales de conexión.

```
============================================================
   STOCKPULSE — Pipeline de datos
============================================================

>>> PASO 1: Ingesta de datos
[INGESTA] Cargando XLSX: data/raw/Online Retail Data Set.xlsx
[INGESTA] Dataset cargado: 541909 filas, 8 columnas.
[INGESTA] Validación de columnas OK.

>>> PASO 2: Limpieza y normalización
[LIMPIEZA] Iniciando con 541909 filas.
[LIMPIEZA] Tras eliminar cancelaciones: 532621 filas.
[LIMPIEZA] Tras filtrar Quantity > 0: 531285 filas.
[LIMPIEZA] Tras filtrar UnitPrice > 0: 530104 filas.
[LIMPIEZA] Duplicados eliminados: 5226.
[LIMPIEZA] Nulos gestionados en Description y CustomerID.
[LIMPIEZA] Fechas normalizadas a formato DATE.
[LIMPIEZA] Texto normalizado en Description y StockCode.
[LIMPIEZA] Pipeline completado. Filas finales: 524878 (de 541909 originales).
[LIMPIEZA] Filas eliminadas en total: 17031.

>>> PASO 3: Separación en tablas Productos / Ventas
[SPLIT] Productos únicos: 3812
[SPLIT] Registros de ventas: 524878

>>> PASO 4: Guardando dataset limpio para módulo ML
[PIPELINE] Dataset limpio guardado en: data\processed\ventas_limpias.csv
```

Base de datos Supabase con los datos cargados mediante supabase-py en la fase inicial. Las tablas productos y ventas contienen los registros del dataset procesado. Se muestran solo algunas filas.

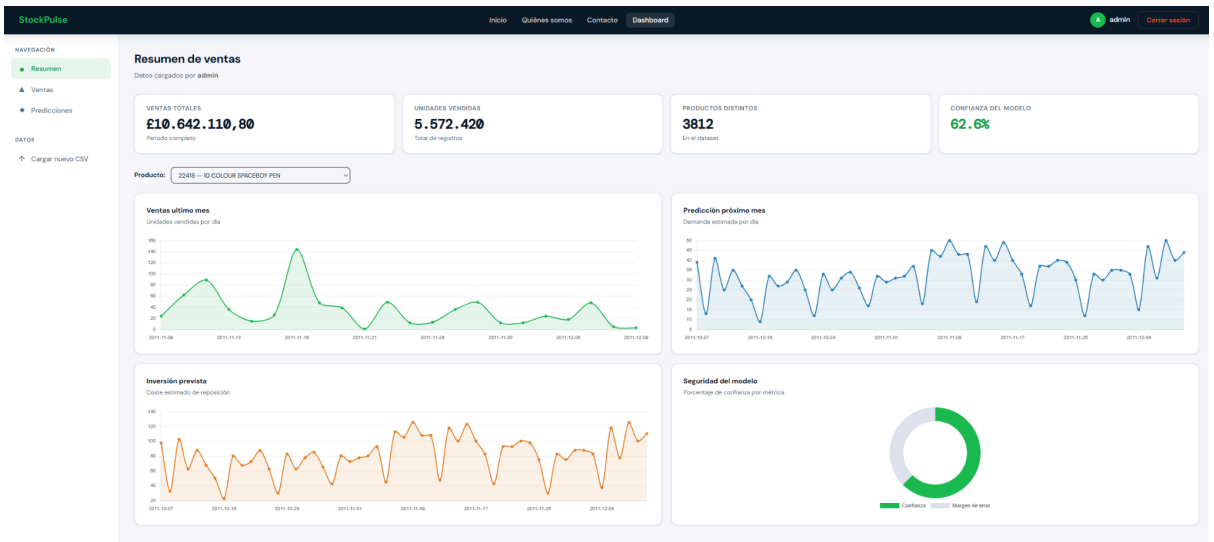
id_producto	nombre varchar	categoria varchar	precio_unitario numeric
1	White Hanging Heart T-Light Holder	General	2.55
2	White Metal Lantern	General	3.39
3	Cream Cupid Hearts Coat Hanger	General	2.75
4	Knitted Union Flag Hot Water Bottle	General	3.39
5	Red Woolly Hottie White Heart.	General	3.39
6	Set 7 Babushka Nesting Boxes	General	7.65
7	Glass Star Frosted T-Light Holder	General	4.25
8	Hand Warmer Union Jack	General	1.85
9	Hand Warmer Red Polka Dot	General	1.85
10	Assorted Colour Bird Ornament	General	1.69
11	Poppy'S Playhouse Bedroom	General	2.10
12	Poppy'S Playhouse Kitchen	General	2.10
13	Feltcraft Princess Charlotte Doll	General	3.75
14	Ivory Knitted Mug Cosy	General	1.65
15	Box Of 6 Assorted Colour Teaspoons	General	4.25
16	Box Of Vintage Jigsaw Blocks	General	4.95
17	Box Of Vintage Alphabet Blocks	General	9.95
18	Home Building Block Word	General	5.95
19	Love Building Block Word	General	5.95
20	Recipe Box With Metal Heart	General	7.95

id_venta i	id_producto	nombre varchar	fecha_venta date	unidades_vendidas int4	total_venta numeric
1	1	NULL	2010-12-01	6	15.30
2	2	NULL	2010-12-01	6	20.34
3	3	NULL	2010-12-01	8	22.00
4	4	NULL	2010-12-01	6	20.34
5	5	NULL	2010-12-01	6	20.34
6	6	NULL	2010-12-01	2	15.30
7	7	NULL	2010-12-01	6	25.50
8	8	NULL	2010-12-01	6	11.10
9	9	NULL	2010-12-01	6	11.10
10	10	NULL	2010-12-01	32	54.08
11	11	NULL	2010-12-01	6	12.60
12	12	NULL	2010-12-01	6	12.60
13	13	NULL	2010-12-01	8	30.00
14	14	NULL	2010-12-01	6	9.90
15	15	NULL	2010-12-01	6	25.50
16	16	NULL	2010-12-01	3	14.85
17	17	NULL	2010-12-01	2	19.90
18	18	NULL	2010-12-01	3	17.85
19	19	NULL	2010-12-01	3	17.85
20	20	NULL	2010-12-01	4	31.80

Entrenamiento del modelo Ridge Regression. Métricas sobre el conjunto de test: RMSE 5069.20, MAE 4355.06, R^2 0.6262. El modelo explica aproximadamente el 63% de la varianza de las ventas diarias.

```
Cargando dataset: data\processed\ventas_limpias.csv
Serie temporal: 305 dias (2010-12-01 -> 2011-12-09)

--- Preprocesamiento ---
Tratando outliers:
  Outliers detectados: 11
  Rango permitido: [-4086, 38150]
Aplicando transformacion log:
  Skewness antes del log: 0.63
  Skewness despues del log: -0.83
Train: 220 dias | Test: 55 dias

Entrenando Ridge (alpha=10.0)...

--- Resultados modelo definitivo ---
  RMSE: 5069.20
  MAE:  4355.06
  R2:   0.6262

Modelo guardado en: src\models\saved\ridge_model.pkl
Scaler guardado en: src\models\saved\scaler.pkl
Predicciones guardadas en: data\results\predicciones_ridge.csv
Grafica guardada en: data\results\prediccion_ridge.png

Todo listo!
```

Dashboard de StockPulse con datos reales cargados desde Supabase. Se muestran los cuatro KPIs principales: ventas totales, unidades vendidas, productos distintos y confianza en el modelo.



4. Arquitectura del sistema implementada

Este apartado describe la arquitectura técnica real del sistema StockPulse tal como ha sido implementada en la Fase 2, comparándola con el diseño planteado en la Fase 1.

4.1 Cambio tecnológico: MySQL → Supabase (PostgreSQL)

Durante el desarrollo de la Fase 2 se identificó una limitación importante en el diseño original: MySQL con XAMPP requiere que un servidor local esté en ejecución en la máquina del desarrollador para que el resto del equipo pueda conectarse. Esto generaba una dependencia de infraestructura incompatible con el trabajo en remoto.

La solución adoptada fue migrar a Supabase, un servicio de base de datos PostgreSQL en la nube con capa gratuita. Esta decisión fue validada con el profesor, quien autorizó el cambio solicitando que se documentara el motivo.

Criterio	MySQL + XAMPP (Fase 1)	Supabase PostgreSQL (Fase 2)
Acceso remoto	No (solo local)	Sí (URL pública)
Coste	Gratuito	Gratuito (capa free)
Sintaxis SQL	MySQL	PostgreSQL (muy similar)
Integración Python	mysql-connector	psycopg2
Compatibilidad equipo	Requiere XAMPP local	Acceso compartido inmediato

4.2 Capas del sistema

La arquitectura de StockPulse se organiza en cinco capas funcionales independientes que se comunican de forma secuencial:

Capa	Función	Tecnología	Responsable
Ingesta	Carga CSV/XLSX	Python + Pandas	Sergio Romero
Procesamiento (ETL)	Limpieza y normalización	Python + Pandas	Sergio Romero
Almacenamiento	Persistencia estructurada	Supabase (PostgreSQL)	Sergio + Joaquín
Modelado ML	Predicción de demanda	Scikit-learn / Prophet	Deninson Tapia
Visualización	Dashboard interactivo	Streamlit / Power BI	Jimmy Chen + Joaquín

4.3 Flujo de datos

El flujo de datos dentro del sistema sigue la siguiente secuencia lineal:

1. **Ingesta:** El Data Engineer ejecuta run_pipeline.py pasando la ruta al dataset (CSV o XLSX).
2. **Carga:** ingest.py carga el archivo, valida las columnas obligatorias y devuelve un DataFrame en bruto.
3. **Limpieza:** clean.py aplica todas las operaciones de limpieza y normalización (eliminación de cancelaciones, duplicados, nulos, normalización de fechas y texto). Devuelve el DataFrame limpio.
4. **Separación:** split_productos_ventas() separa el DataFrame en dos tablas: Productos (catálogo único) y Ventas (transacciones).
5. **Exportación:** El dataset limpio se guarda como CSV en data/processed/ para uso del módulo de ML.
6. **Carga BD:** load_db.py crea las tablas si no existen e inserta los datos en Supabase por lotes de 5.000 filas.
7. **Modelado:** El módulo de predicción (Deninson) lee desde la BD, entrena el modelo y escribe en la tabla predicciones.
8. **Visualización:** El dashboard (Jimmy) consulta la BD y muestra ventas reales vs. predicciones.

4.4 Separación de responsabilidades

Cada módulo del pipeline es independiente y puede modificarse sin afectar al resto. La comunicación entre módulos se realiza exclusivamente a través de interfaces bien definidas (DataFrames de Pandas y tablas de la BD):

- src/pipeline/ingest.py → carga y validación de datos de entrada
- src/pipeline/clean.py → limpieza, normalización y split de tablas
- src/pipeline/load_db.py → conexión, creación de tablas e inserción en Supabase

- src/pipeline/run_pipeline.py → orquestador que encadena los tres módulos anteriores

5. Índice del código — Estructura del proyecto

A continuación se detalla la estructura real del repositorio al final de la Fase 2, con una descripción de la función de cada archivo:

```
Proyecto_IA_BigData_Grupo5/

├── docs/                                   ← Documentación del proyecto
│   ├── Fase0_Presentacion_alineacion_viabilidad_Grupo5.pdf
│   └── Fase1_Grupo5.pdf

├── environment/                            ← Configuración del entorno
│   └── requirements.txt                    ← Dependencias del proyecto

├── src/                                    ← Código fuente del proyecto
│   ├── backend/                            ← Módulo Backend (FastAPI)
│   │   ├── routes/                         ← Endpoints REST
│   │   │   ├── __init__.py
│   │   │   ├── auth_routes.py              ← Endpoints de autenticación
│   │   │   ├── dashboard_routes.py         ← Endpoints del dashboard
│   │   │   └── upload_routes.py            ← Endpoint de carga del pipeline
│   │   ├── auth.py                         ← Gestión de sesiones y tokens
│   │   ├── database.py                     ← Conexión a Supabase (PostgreSQL)
│   │   └── main.py                         ← Punto de entrada FastAPI
│   ├── frontend/                           ← Módulo Frontend
│   │   ├── app.js                          ← Lógica del dashboard (JS)
│   │   ├── index.html                      ← Interfaz principal del proyecto
│   │   └── styles.css                      ← Estilos de la interfaz
│   ├── models/                             ← Módulo Data Scientist
│   │   ├── features.py                     ← Ingeniería de variables (features)
│   │   └── train_model.py                  ← Entrenamiento y predicciones
│   └── pipeline/                           ← Módulo Data Engineer (Sergio Romero)
│   ├── __init__.py
│   ├── ingest.py                           ← Capa de ingesta (carga CSV/XLSX)
│   ├── clean.py                            ← Capa ETL (limpieza + normalización)
│   ├── load_db.py                          ← Capa de almacenamiento (Supabase)
│   └── run_pipeline.py                     ← Orquestador del pipeline

├── .gitignore                              ← Archivos excluidos del repositorio
└── README.md                               ← Descripción del proyecto
```

6. Código completo del proyecto

Indice de archivos

- `README.md`
- `.gitignore`
- `environment/requirements.txt`
- `src/backend/main.py`
- `src/backend/auth.py`
- `src/backend/database.py`
- `src/backend/routes/__init__.py`
- `src/backend/routes/auth_routes.py`
- `src/backend/routes/dashboard_routes.py`
- `src/backend/routes/upload_routes.py`
- `src/frontend/index.html`
- `src/frontend/styles.css`
- `src/frontend/app.js`
- `src/models/features.py`
- `src/models/train_model.py`
- `src/pipeline/__init__.py`
- `src/pipeline/ingest.py`
- `src/pipeline/clean.py`
- `src/pipeline/load_db.py`
- `src/pipeline/run_pipeline.py`

Archivo: README.md

Ruta completa: `README.md`

```
StockPulse

Proyecto de Big Data e Inteligencia Artificial

Descripción del proyecto

StockPulse es un sistema de análisis y predicción de ventas diseñado para ayudar a
mejorar la gestión de inventario en empresas.
El sistema analiza datos históricos de ventas para identificar patrones de
comportamiento y generar predicciones de demanda futura. Estas predicciones se
visualizan mediante un dashboard interactivo que permite comparar ventas reales con
estimaciones y facilitar la toma de decisiones.
El objetivo principal del proyecto es reducir problemas comunes en la gestión de
inventario como:
   -  exceso de stock
   -  roturas de inventario
   -  planificación incorrecta de compras

Para ello se implementará un pipeline de datos que permitirá procesar datasets de
ventas, entrenar un modelo de predicción y visualizar los resultados de forma
clara.
```

Integrantes

Grupo 5

- Sergio Romero — Data Engineer
  Responsable de ingesta de datos, limpieza del dataset y análisis exploratorio (EDA).
- Deninson Tapia — Data Analyst / Data Scientist
  Responsable del modelado de datos, entrenamiento del modelo y evaluación de resultados.
- Joaquín Fernández — Platform / Project Manager
  Responsable del repositorio, API, integración de componentes y despliegue.
- Jimmy Chen — Business Intelligence
  Responsable del dashboard, visualización de datos y definición de KPIs.

Arquitectura del sistema (resumen)

El sistema sigue una arquitectura basada en un pipeline de datos dividido en varias capas:

1. Ingesta de datos
- Carga de datasets de ventas en formato CSV o XLSX.
2. Procesamiento de datos
- Limpieza de datos
- Eliminación de duplicados
- Normalización de fechas y categorías
3. Almacenamiento
- Los datos procesados se almacenan en una base de datos MySQL.
4. Modelado
- Se entrena un modelo de predicción de demanda utilizando librerías de Machine Learning.
5. Visualización
- Un dashboard mostrará métricas, tendencias y predicciones de ventas.

Flujo simplificado: Dataset → Limpieza de datos → Base de datos → Modelo predictivo → Dashboard

---

Tecnologías utilizadas

El proyecto utiliza tecnologías comunes en el ecosistema de Big Data y Data Science:

Lenguaje principal
- Python

Procesamiento de datos
- Pandas
- NumPy

Machine Learning
- Scikit-learn
- Prophet (alternativa para series temporales)

Base de datos
- PostgreSQL (Supabase)

Visualización
- Uvicorn

Herramientas de desarrollo
- Git
- GitHub
- Jupyter Notebook

Organización del repositorio

```
repo/
│
├── docs/
│   ├── Fase0_Presentacion_y_Viabilidad.pdf
│   ├── Fase1_Grupo5.pdf
```

```
│   └── Fase2_Producto_Funcional_Local_y_Codigo_Grupo5.pdf
├── environment/
│   └── requirements.txt
├── src/
│   ├── backend
│   │   ├── routes
│   │   │   ├── __init__.py
│   │   │   ├── auth_routes.py
│   │   │   ├── dashboard_routes.py
│   │   │   └── upload_routes.py
│   │   ├── auth.py
│   │   ├── database.py
│   │   └── main.py
│   ├── frontend
│   │   ├── app.js
│   │   ├── index.html
│   │   └── styless.css
│   ├── models
│   │   ├── features.py
│   │   └── train_model.py
│   └── pipeline
│       ├── __init__.py
│       ├── clean.py
│       ├── ingest.py
│       ├── load_db.py
│       └── run_pipeline.py
├── README.md
└── .gitignore
```

📁 docs/
Contiene la documentación del proyecto:

- Entregables del curso
- Diagramas de arquitectura
- Memoria técnica
- Presentaciones

📁 environment/
Contiene la configuración del entorno:

- `requirements.txt`
- configuración de dependencias

📁 src/
Contiene el código del proyecto:

- Scripts de limpieza de datos
- Scripts de entrenamiento del modelo
- Evaluación del modelo
- API o scripts de integración
- Código del dashboard

⚠️ Los datasets grandes no se subirán al repositorio.
Solo se incluirán muestras o instrucciones de descarga.

Cómo ejecutar el proyecto
⚠️ El sistema aún se encuentra en desarrollo, pero la estructura básica de ejecución será la siguiente.
1. Clonar el repositorio
git clone https://github.com/Joaquin-AI-DEV/Proyecto_IA_BigData_Grupo5.git
cd Proyecto_IA_BigData_Grupo5

2. Crear y activar entorno virtual
Crear entorno virtual
python -m venv venv
Activar en Windows
venv\Scripts\activate
Activar en Linux / Mac
source venv/bin/activate

3. Instalar dependencias
pip install -r environment/requirements.txt

4. Configurar variables del entorno
Copia el archivo de ejemplo y rellena las credenciales de Supabase:
Linux / Mac
cp environment/.env.example environment/.env

Windows
copy environment\.env.example environment\.env

# El archivo .env debe contener:
DB_HOST=aws-0-eu-west-1.pooler.supabase.com
DB_PORT=6543
DB_NAME=postgres
DB_USER=postgres.nbamabcfxtcltbzxuncg
DB_PASSWORD=123Grupo5321
5. Ejecución del proyecto Carga de CSV y Inicio de sesión
Ejecutar el comando: uvicorn main:app --reload --port 8000 La máquina virtual debe estar encendida y las dependencias instaladas, debes encontrarte en la carpeta src/backend Verás la interfaz de inicio del proyecto, deberás inciar sesión con las credenciales: Usuario: admin Contraseña: admin123

Estado del proyecto
El proyecto está en fase de pulido, durante la seguiente Fase se introducirán cambios en la visualización, métricas, resultados, métodos y accesibilidad del proyecto.

Archivo: .gitignore

Ruta completa: `.gitignore`

```
# Entornos virtuales
venv/
venvProyectoFP/
.venv/
env/
```

```
# Variables de entorno (credenciales Supabase)
environment/.env
.env

# Datos - archivos grandes no se suben al repo
backend/data/raw/*.xlsx
backend/data/raw/*.csv
backend/data/processed/
backend/data/results/
data/raw/*.xlsx
data/raw/*.csv
data/processed/
data/results/

# Modelos entrenados (se regeneran con train_model.py)
src/models/saved/

# Python
__pycache__/
*.pyc
*.pyo
.ipynb_checkpoints/

# IDEs
.vscode/
.idea/

# Sistema
.DS_Store
Thumbs.db
```

Archivo: requirements.txt

Ruta completa: `environment/requirements.txt`

```
# StockPulse — Dependencias del proyecto
# Instalación: pip install -r environment/requirements.txt
# Python recomendado: 3.10+

# --- Procesamiento de datos ---
pandas==2.2.2
numpy==1.26.4
openpyxl==3.1.2            # Lectura de archivos XLSX

# --- Base de datos (Supabase / PostgreSQL) ---
psycopg2-binary==2.9.9  # Conector PostgreSQL
python-dotenv==1.0.1       # Lectura de variables de entorno (.env)

# --- Machine Learning (módulo Deninson) ---
scikit-learn==1.4.2
prophet==1.1.5

# --- Visualización / EDA (Jupyter) ---
```

```
matplotlib==3.8.4
seaborn==0.13.2
jupyter==1.0.0
ipykernel==6.29.4

# --- Dashboard (módulo Jimmy) ---
streamlit==1.35.0

# --- Backend API (módulo Sergio) ---
fastapi==0.111.0           # Framework de API REST
uvicorn==0.30.1            # Servidor ASGI para FastAPI
python-multipart==0.0.9 # Necesario para recibir archivos (Form + UploadFile)
bcrypt==4.1.3              # Hash seguro de contraseñas
```

Archivo: main.py

Ruta completa: `src/backend/main.py`

```
# main.py — Punto de entrada del backend FastAPI
# Proyecto: StockPulse
# Autor: Sergio Romero (Data Engineer)

# Responsabilidad:
# Configura la aplicación FastAPI, registra todos los routers,
# habilita CORS y sirve el frontend (index.html) directamente
# desde http://localhost:8000

# Uso:
# Desde la carpeta src/backend/:
# uvicorn main:app -reload -port 8000


# Luego abrir http://localhost:8000 en el navegador — carga el index.html
automáticamente.



import os
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

# Importar los routers de cada módulo

from routes.auth_routes import router as auth_router
from routes.upload_routes import router as upload_router
from routes.dashboard_routes import router as dashboard_router

# -- Crear la aplicación --

app = FastAPI(
title="StockPulse API",
description="Backend del sistema de predicción de demanda StockPulse.",
version="1.0.0",
```

```
)

# -- CORS --

# Necesario para que el navegador permita llamadas desde el frontend

# al backend. En producción se limitaría a la URL del frontend real.

app.add_middleware(
CORSMiddleware,
allow_origins=["*"],       # En desarrollo permitimos cualquier origen
allow_credentials=True,
allow_methods=["*"],
allow_headers=["*"],
)

# -- Registrar routers de la API --

app.include_router(auth_router)         # /api/auth/login, /api/auth/logout
app.include_router(upload_router)       # /api/upload
app.include_router(dashboard_router)  # /api/dashboard/…

# -- Servir el frontend --

# Ruta a la carpeta frontend relativa a src/backend/

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FRONTEND_DIR = os.path.join(BASE_DIR, "..", "frontend")

# Montar archivos estáticos (styles.css, app.js) bajo /static

app.mount("/static", StaticFiles(directory=FRONTEND_DIR), name="static")

@app.get("/")
def serve_frontend():

# Sirve el index.html al abrir http://localhost:8000 en el navegador.
# Así no hace falta abrir el HTML manualmente — uvicorn lo sirve todo.

       return FileResponse(os.path.join(FRONTEND_DIR, "index.html"))
```

Archivo: auth.py

Ruta completa: `src/backend/auth.py`

```
"""
auth.py — Gestión de sesiones y tokens
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Responsabilidad:
    Gestiona los tokens de sesión activos en memoria.
       Cuando el usuario hace login se genera un token UUID único.
       Cuando hace logout el token se elimina, invalidando la sesión.
```

```
    Además, cuando el usuario cierra sesión, se borran todos los
    registros de ventas y productos que subió durante su sesión
       (comportamiento solicitado por Joaquín en el Discord).

      NOTA: Este sistema de sesiones en memoria es suficiente para
    el alcance del proyecto (demo académica). En producción se
    usaría JWT con firma o almacenamiento de sesiones en Redis.
"""

import uuid
import bcrypt
from database import get_connection


# Almacén de sesiones activas: { token: username }
# Se pierde al reiniciar el servidor, lo cual es el comportamiento deseado
_active_sessions: dict[str, str] = {}


def verify_password(plain_password: str, hashed_password: str) -> bool:
       """
    Verifica si una contraseña en texto plano coincide con su hash bcrypt.

    Parámetros:
        plain_password (str): Contraseña introducida por el usuario.
        hashed_password (str): Hash almacenado en la base de datos.

    Retorna:
       bool: True si coinciden, False si no.
       """
       return bcrypt.checkpw(
        plain_password.encode("utf-8"),
        hashed_password.encode("utf-8")
       )


def authenticate_user(username: str, password: str) -> bool:
       """
       Consulta la tabla 'usuarios' en Supabase y verifica las credenciales.

    Parámetros:
       username (str): Nombre de usuario.
       password (str): Contraseña en texto plano.

    Retorna:
       bool: True si las credenciales son correctas, False si no.
       """
       try:
       conn = get_connection()
       with conn.cursor() as cur:
             cur.execute(
              "SELECT password FROM usuarios WHERE username = %s",
              (username,)
              )
              row = cur.fetchone()
        conn.close()

       if not row:
              return False  # Usuario no existe
```

```
        return verify_password(password, row["password"])

        except Exception as e:
        print(f"[AUTH] Error al autenticar usuario: {e}")
        return False


def create_session(username: str) -> str:
        """
        Crea una nueva sesión para el usuario y devuelve el token generado.

    Parámetros:
        username (str): Nombre del usuario que ha iniciado sesión.

    Retorna:
        str: Token UUID único de sesión.
        """
        token = str(uuid.uuid4())
        _active_sessions[token] = username
        print(f"[AUTH] Sesión creada para '{username}' — token: {token[:8]}...")
        return token


def get_session_user(token: str) -> str | None:
        """
    Devuelve el nombre de usuario asociado a un token activo.

    Parámetros:
        token (str): Token de sesión del cliente.

    Retorna:
        str | None: Username si la sesión existe, None si no.
        """
        return _active_sessions.get(token)


def destroy_session(token: str) -> str | None:
        """
    Elimina la sesión activa y limpia los datos de BD del usuario.
        Esta es la lógica de 'cerrar sesión borra los datos' que pidió Joaquín.

    Parámetros:
        token (str): Token de sesión a invalidar.

    Retorna:
        str | None: Username si la sesión existía, None si no.
        """
        username = _active_sessions.pop(token, None)

        if username:
        print(f"[AUTH] Sesión destruida para '{username}'.")
        _clear_user_data()

        return username


def _clear_user_data() -> None:
        """
```

```
        Borra todos los registros de ventas, productos y predicciones de la BD.
        Se llama automáticamente al cerrar sesión.

        NOTA: Borra toda la BD porque el sistema es monousuario en esta fase.
        Cuando haya múltiples usuarios reales, habría que filtrar por id_usuario.
        """
        try:
        conn = get_connection()
        with conn.cursor() as cur:
              # Borrar en orden inverso a las FK para evitar violaciones de
restricción
            cur.execute("DELETE FROM predicciones;")
            cur.execute("DELETE FROM ventas;")
            cur.execute("DELETE FROM productos;")
         conn.commit()
         conn.close()
        print("[AUTH] Datos de la BD borrados al cerrar sesión.")
        except Exception as e:
        print(f"[AUTH] Error al limpiar datos de BD: {e}")


def is_valid_token(token: str) -> bool:
        """
    Comprueba si un token de sesión es válido.

    Parámetros:
        token (str): Token a verificar.

    Retorna:
        bool: True si el token es válido, False si no.
        """
        return token in _active_sessions
```

Archivo: database.py

Ruta completa: `src/backend/database.py`

```
"""
database.py — Gestión de conexión a Supabase (PostgreSQL)
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Responsabilidad:
    Provee una función de conexión centralizada para que todos los
    módulos del backend usen la misma configuración sin repetir código.
        Las credenciales se cargan desde el archivo .env en la raíz del proyecto.
"""

import os
from pathlib import Path
import psycopg2
from psycopg2.extras import RealDictCursor
from dotenv import load_dotenv

# Resolver la ruta al .env de forma independiente al CWD.
```

```
# Estructura: <proyecto>/src/backend/database.py  →  <proyecto>/environment/.env
ENV_PATH = Path(__file__).resolve().parents[2] / "environment" / ".env"
load_dotenv(dotenv_path=ENV_PATH)


def get_connection():
       """
       Crea y devuelve una conexión a PostgreSQL en Supabase.

    Retorna:
       psycopg2.connection: Conexión activa con cursor tipo diccionario.

    Excepciones:
        RuntimeError: Si faltan variables obligatorias en el .env.
       psycopg2.OperationalError: Si no hay conexión o las credenciales son
incorrectas.
       """
       host   = os.getenv("DB_HOST")
       user   = os.getenv("DB_USER")
       password = os.getenv("DB_PASSWORD")

       if not (host and user and password):
       raise RuntimeError(
            f"Faltan variables en {ENV_PATH}. Requeridas: DB_HOST, DB_USER,
DB_PASSWORD. "
             "Copia environment/.env.example a environment/.env y rellénalo."
       )

       return psycopg2.connect(
       host=host,
       port=int(os.getenv("DB_PORT", 6543)),
        dbname=os.getenv("DB_NAME", "postgres"),
       user=user,
       password=password,
        sslmode="require",
        cursor_factory=RealDictCursor,
       )
```

Archivo: __init__.py

Ruta completa: `src/backend/routes/__init__.py`

```
# Paquete de rutas del backend StockPulse
```

Archivo: auth_routes.py

Ruta completa: `src/backend/routes/auth_routes.py`

```
"""
routes/auth_routes.py — Endpoints de autenticación
Proyecto: StockPulse
```

```
Autor: Sergio Romero (Data Engineer)

Endpoints:
       POST /api/auth/login   → Valida credenciales y devuelve token
       POST /api/auth/logout  → Invalida token y borra datos de BD
"""

from fastapi import APIRouter, HTTPException, Header
from pydantic import BaseModel
from auth import authenticate_user, create_session, destroy_session

router = APIRouter(prefix="/api/auth", tags=["Autenticación"])


# ---- Modelos de request ----

class LoginRequest(BaseModel):
       """Cuerpo de la petición de login."""
       username: str
       password: str


# ---- Endpoints ----

@router.post("/login")
def login(body: LoginRequest):
       """
    Autentica al usuario contra la tabla 'usuarios' de Supabase.
       Si las credenciales son correctas, devuelve un token de sesión.

    Retorna:
       {
              "token": "uuid-del-token",
              "username": "admin"
       }
       """
       # Validar que los campos no estén vacíos
       if not body.username.strip() or not body.password:
       raise HTTPException(status_code=400, detail="Rellena todos los campos.")

       # Verificar credenciales contra la BD
       if not authenticate_user(body.username.strip(), body.password):
       raise HTTPException(status_code=401, detail="Usuario o contraseña
incorrectos.")

       # Crear sesión y devolver token
       token = create_session(body.username.strip())
       return {"token": token, "username": body.username.strip()}


@router.post("/logout")
def logout(authorization: str = Header(None)):
       """
       Cierra la sesión del usuario.
    Invalida el token y borra los datos de la BD (comportamiento solicitado).

       Header requerido:
       Authorization: Bearer <token>
       """
```

```
        token = _extract_token(authorization)

        username = destroy_session(token)

        if not username:
        # Si el token no existía, simplemente respondemos OK
        # (el frontend puede llamar a logout aunque la sesión ya expiró)
        return {"message": "Sesión cerrada."}

        return {"message": f"Sesión de '{username}' cerrada. Datos eliminados."}


# ---- Helper ----

def _extract_token(authorization: str | None) -> str:
        """
    Extrae el token del header 'Authorization: Bearer <token>'.

    Excepciones:
        HTTPException 401: Si el header no existe o tiene formato incorrecto.
        """
        if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Token de sesión requerido.")
        return authorization.split(" ", 1)[1]
```

Archivo: dashboard_routes.py

Ruta completa: `src/backend/routes/dashboard_routes.py`

```
"""
routes/dashboard_routes.py — Endpoints de datos para el dashboard
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Endpoints:
        GET /api/dashboard/kpis                 → KPIs globales (ventas totales,
unidades, productos)
        GET /api/dashboard/productos            → Lista de productos para el
selector
        GET /api/dashboard/ventas/{id_producto}  → Ventas del último mes por
producto
        GET /api/dashboard/predicciones/{id}     → Predicciones por producto (de la
tabla predicciones)

Estos endpoints son los que Jimmy Chen necesita para alimentar los gráficos del
frontend.
"""
```

```python
from fastapi import APIRouter, HTTPException, Header
from database import get_connection
from auth import get_session_user

router = APIRouter(prefix="/api/dashboard", tags=["Dashboard"])


# ---- Helpers ----

def _extract_token(authorization: str | None) -> str:
       """Extrae y valida el token del header Authorization."""
       if not authorization or not authorization.startswith("Bearer "):
       raise HTTPException(status_code=401, detail="Token de sesión requerido.")
       return authorization.split(" ", 1)[1]


def _require_session(authorization: str | None) -> str:
       """Valida que la sesión sea activa. Retorna el username."""
       token = _extract_token(authorization)
       user = get_session_user(token)
       if not user:
       raise HTTPException(status_code=401, detail="Sesión no válida o expirada.")
       return user


# ---- Endpoints ----

@router.get("/kpis")
def get_kpis(authorization: str = Header(None)):
       """
    Devuelve los KPIs globales para las tarjetas del dashboard:
       - Total de ventas (suma de total_venta)
       - Total de unidades vendidas
       - Número de productos distintos en el dataset

    Retorna:
       {
              "ventas_totales": 8887.25,
              "unidades_totales": 521243,
              "productos_distintos": 3684
       }
       """
       _require_session(authorization)

       try:
       conn = get_connection()
       with conn.cursor() as cur:
             cur.execute("""
              SELECT
                     ROUND(SUM(total_venta)::numeric, 2)  AS ventas_totales,
                     SUM(unidades_vendidas)                  AS unidades_totales
              FROM ventas;
              """)
             row_ventas = cur.fetchone()

             cur.execute("SELECT COUNT(*) AS total FROM productos;")
             row_productos = cur.fetchone()
```

```
        conn.close()

       return {
              "ventas_totales": float(row_ventas["ventas_totales"] or 0),
              "unidades_totales": int(row_ventas["unidades_totales"] or 0),
              "productos_distintos": int(row_productos["total"] or 0),
       }

       except Exception as e:
       raise HTTPException(status_code=500, detail=f"Error al obtener KPIs:
{str(e)}")


@router.get("/productos")
def get_productos(authorization: str = Header(None)):
       """
    Devuelve la lista de productos disponibles para el selector del dashboard.

    Retorna:
       [
             { "id_producto": "85123A", "nombre": "WHITE HANGING HEART..." },
              ...
       ]
       """
       _require_session(authorization)

       try:
       conn = get_connection()
       with conn.cursor() as cur:
             cur.execute("""
              SELECT id_producto, nombre
              FROM productos
              ORDER BY nombre
              LIMIT 500;
              """)
              rows = cur.fetchall()
        conn.close()

       return [dict(row) for row in rows]

       except Exception as e:
       raise HTTPException(status_code=500, detail=f"Error al obtener productos:
{str(e)}")


@router.get("/ventas/{id_producto}")
def get_ventas_producto(id_producto: str, authorization: str = Header(None)):
       """
    Devuelve las ventas diarias del último mes para un producto concreto.
    Utilizado por el gráfico 'Ventas último mes' del dashboard.

    Parámetros:
        id_producto (str): Código del producto (StockCode del dataset).

    Retorna:
       {
              "fechas": ["2011-11-01", "2011-11-02", ...],
              "unidades": [45, 30, 62, ...]
       }
```

```
        """
        _require_session(authorization)

        try:
        conn = get_connection()
        with conn.cursor() as cur:
              cur.execute("""
               SELECT
                      fecha_venta,
                      SUM(unidades_vendidas) AS unidades
               FROM ventas
               WHERE id_producto = %s
                      AND fecha_venta >= (
                      SELECT MAX(fecha_venta) - INTERVAL '30 days'
                      FROM ventas
                      WHERE id_producto = %s
                      )
               GROUP BY fecha_venta
               ORDER BY fecha_venta;
               """, (id_producto, id_producto))
               rows = cur.fetchall()
         conn.close()

        return {
               "fechas":   [str(row["fecha_venta"]) for row in rows],
               "unidades": [int(row["unidades"]) for row in rows],
        }

        except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error al obtener ventas:
{str(e)}")


@router.get("/predicciones/{id_producto}")
def get_predicciones_producto(id_producto: str, authorization: str = Header(None)):
        """
    Devuelve las predicciones generadas por el modelo ML (Deninson)
        para un producto concreto. Alimenta el gráfico 'Predicción próximo mes'.

    Parámetros:
         id_producto (str): Código del producto.

    Retorna:
        {
               "fechas":     ["2011-12-01", "2011-12-02", ...],
               "predichas":  [50, 43, 67, ...],
               "confianza":  0.87
        }
        """
        _require_session(authorization)

        try:
        conn = get_connection()
        with conn.cursor() as cur:
              cur.execute("""
               SELECT
                      fecha_prediccion,
                      unidades_predichas,
                      confianza_modelo
```

```
            FROM predicciones
            WHERE id_producto = %s
            ORDER BY fecha_prediccion;
            """, (id_producto,))
            rows = cur.fetchall()
        conn.close()

    if not rows:
            # El modelo aún no ha generado predicciones para este producto
            return {"fechas": [], "predichas": [], "confianza": None}

    # La confianza del modelo se toma como el promedio de todas las predicciones
        confianza_media = sum(
            float(r["confianza_modelo"]) for r in rows if r["confianza_modelo"]
    ) / len(rows)

    return {
            "fechas":    [str(row["fecha_prediccion"]) for row in rows],
            "predichas": [int(row["unidades_predichas"] or 0) for row in rows],
            "confianza": round(confianza_media, 4),
    }

    except Exception as e:
    raise HTTPException(status_code=500, detail=f"Error al obtener predicciones:
{str(e)}")


# Endpoint para la predicción global (modelo agregado diario, sin id_producto).
# Lo consume el dashboard para el gráfico "ventas reales vs predichas".
@router.get("/predicciones-globales")
def get_predicciones_globales(authorization: str = Header(None)):
    """
    Devuelve la serie temporal de predicciones del modelo agregado (Ridge).
    Cada fila representa un día con sus ventas reales y predichas (sin
producto).

    Retorna:
    {
            "fechas":    ["2011-11-01", ...],
            "reales":    [123, 145, ...],
            "predichas": [130, 140, ...]
    }
    """
    _require_session(authorization)

    try:
    conn = get_connection()
    with conn.cursor() as cur:
            # Filtramos por id_producto IS NULL: así aislamos las predicciones
            # del modelo agregado y no las mezclamos con posibles predicciones
            # per-producto que se añadan en el futuro.
          cur.execute("""
            SELECT fecha_prediccion, unidades_vendidas, unidades_predichas
            FROM predicciones
            WHERE id_producto IS NULL
            ORDER BY fecha_prediccion;
            """)
            rows = cur.fetchall()
        conn.close()
```

```
        return {
                "fechas":    [str(r["fecha_prediccion"]) for r in rows],
                "reales":    [int(r["unidades_vendidas"] or 0) for r in rows],
                "predichas": [int(r["unidades_predichas"] or 0) for r in rows],
        }

        except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error al obtener predicciones
globales: {str(e)}")


@router.get("/inversion/{id_producto}")
def get_inversion_producto(id_producto: str, authorization: str = Header(None)):
        """
    Calcula la inversión prevista para el próximo mes combinando
    predicciones de unidades con el precio unitario del producto.

    Retorna:
        {
                "fechas":    ["2011-12-01", ...],
                "inversion": [125.50, 98.20, ...]
        }
        """
        _require_session(authorization)

        try:
        conn = get_connection()
        with conn.cursor() as cur:
                # Precio unitario del producto
               cur.execute(
                "SELECT precio_unitario FROM productos WHERE id_producto = %s",
                (id_producto,)
                )
                prod = cur.fetchone()

                if not prod:
                return {"fechas": [], "inversion": []}

               precio = float(prod["precio_unitario"])

                # Predicciones del modelo
               cur.execute("""
                SELECT fecha_prediccion, unidades_predichas
                FROM predicciones
                WHERE id_producto = %s
                ORDER BY fecha_prediccion;
                """, (id_producto,))
                rows = cur.fetchall()

         conn.close()

        return {
                "fechas":    [str(r["fecha_prediccion"]) for r in rows],
                "inversion": [round((r["unidades_predichas"] or 0) * precio, 2) for r
in rows],
        }

        except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=f"Error al calcular inversión:
{str(e)}")
```

Archivo: upload_routes.py

Ruta completa: `src/backend/routes/upload_routes.py`

```
"""
routes/upload_routes.py — Endpoint de carga y procesamiento de datos
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Endpoints:
        POST /api/upload → Recibe CSV/XLSX, lanza el pipeline y devuelve resultado

Flujo:
        1. Validar token de sesión
        2. Guardar el archivo temporalmente
        3. Ejecutar run_pipeline() (ingest → clean → load_db)
        4. Eliminar el archivo temporal
        5. Devolver resultado al frontend
"""

import os
import shutil
import tempfile
from fastapi import APIRouter, HTTPException, Header, UploadFile, File, Form
from auth import get_session_user

# Importar el orquestador del pipeline de Sergio
import sys
# Ruta absoluta — funciona independientemente del directorio de trabajo
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(BASE_DIR, "..", "..", ".."))

from src.pipeline.run_pipeline import run_pipeline

router = APIRouter(prefix="/api", tags=["Carga de datos"])

# Extensiones de archivo permitidas
ALLOWED_EXTENSIONS = {".csv", ".xlsx"}
MAX_FILE_SIZE_MB = 50


@router.post("/upload")
async def upload_file(
        file: UploadFile = File(...),
        username: str = Form(...),
        authorization: str = Header(None)
):
        """
    Recibe un archivo CSV o XLSX, ejecuta el pipeline de datos y
    almacena los resultados en Supabase.

    Parámetros (form-data):
```

```
        file     : Archivo CSV o XLSX
         username : Nombre del usuario (para logging)

        Header requerido:
        Authorization: Bearer <token>

    Retorna:
        {
               "message": "Archivo procesado correctamente.",
               "filas_procesadas": 524878,
               "filename": "online_retail.xlsx"
        }
        """

        # 1. Validar sesión
        token = _extract_token(authorization)
    session_user = get_session_user(token)
        if not session_user:
        raise HTTPException(status_code=401, detail="Sesión no válida o expirada.")

        # 2. Validar extensión del archivo
    ext = os.path.splitext(file.filename)[1].lower()
        if ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(
              status_code=400,
               detail=f"Formato no válido: '{ext}'. Solo se permiten .csv y .xlsx"
        )

        # 3. Guardar archivo en directorio temporal del sistema operativo
    tmp_file = None
        try:
        # Crear archivo temporal con la extensión correcta
        with tempfile.NamedTemporaryFile(
               suffix=ext, delete=False, mode="wb"
        ) as tmp:
              tmp_file = tmp.name
               content = await file.read()

               # Validar tamaño (50 MB máximo)
               if len(content) > MAX_FILE_SIZE_MB * 1024 * 1024:
               raise HTTPException(
                      status_code=413,
                      detail=f"Archivo demasiado grande. Máximo
{MAX_FILE_SIZE_MB}MB."
               )

              tmp.write(content)

        print(f"[UPLOAD] Archivo temporal guardado en: {tmp_file}")
        print(f"[UPLOAD] Usuario: {session_user} — Archivo: {file.filename}")

        # 4. Ejecutar el pipeline de datos
        result = run_pipeline(tmp_file)

         # 5. Devolver respuesta al frontend
        return {
               "message": "Archivo procesado correctamente.",
               "filename": file.filename,
        }
```

```
        except HTTPException:
         raise  # Re-lanzar excepciones HTTP que ya preparamos arriba

        except Exception as e:
        print(f"[UPLOAD] Error procesando archivo: {e}")
        raise HTTPException(
             status_code=500,
              detail=f"Error al procesar el archivo: {str(e)}"
        )

        finally:
        # 6. Limpiar siempre el archivo temporal, pase lo que pase
        if tmp_file and os.path.exists(tmp_file):
             os.remove(tmp_file)
              print(f"[UPLOAD] Archivo temporal eliminado: {tmp_file}")


def _extract_token(authorization: str | None) -> str:
        """Extrae el token del header Authorization: Bearer <token>."""
        if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Token de sesión requerido.")
        return authorization.split(" ", 1)[1]
```

Archivo: index.html

Ruta completa: `src/frontend/index.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>StockPulse</title>
  <link rel="stylesheet" href="/static/styles.css" />
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.0/chart.umd.min.js"></scri
pt>
</head>
<body>

  <nav class="navbar">
       <a class="navbar-brand" href="#" onclick="navigateTo('inicio')">
       <span>StockPulse</span>
       </a>

       <ul class="navbar-links" id="navLinks">
```

```
      <li><a href="#" onclick="navigateTo('inicio')" class="active"
data-page="inicio">Inicio</a></li>
      <li><a href="#" onclick="navigateTo('quienes')" data-page="quienes">Quiénes
somos</a></li>
      <li><a href="#" onclick="navigateTo('contacto')"
data-page="contacto">Contacto</a></li>
     <!-- Solo visible con sesión -->
      <li id="navDashboardItem" style="display:none;">
      <a href="#" onclick="navigateTo('dashboard')"
data-page="dashboard">Dashboard</a>
      </li>
      </ul>

      <div class="navbar-actions" id="navActions">
     <!-- Sin sesión -->
      <div id="navGuest">
      <button class="btn btn-primary" onclick="openLoginModal()">Iniciar
sesión</button>
      </div>
     <!-- Con sesión -->
      <div id="navUser" style="display:none;" class="navbar-user">
      <div class="navbar-avatar" id="navAvatar">A</div>
      <span id="navUsername">admin</span>
      <button class="btn btn-danger" onclick="logout()"
style="margin-left:0.5rem;">Cerrar sesión</button>
      </div>
      </div>
  </nav>

  <!-- INICIO -->
  <div class="page active" id="page-inicio">
      <section class="hero">
      <h2 class="hero-title">StockPulse</h2>
      <span class="hero-badge">Sistema de predicción de demanda</span>
      <h1>Anticipa tu <span>stock</span>,<br>optimiza tu negocio</h1>
      <p>Carga tus datos de ventas y obtén predicciones precisas de demanda con
modelos de machine learning.</p>

      <div class="hero-actions">
      <button class="btn btn-primary btn-lg" onclick="handleUploadClick()">
         Cargar CSV / XLSX
      </button>
      <button class="btn btn-outline btn-lg" onclick="navigateTo('quienes')">
      Saber más
      </button>
      </div>

     <!-- Upload area (visible solo con sesión) -->
      <div class="upload-area" id="uploadArea" style="display:none;"
onclick="triggerFileInput()">
      <input type="file" id="fileInput" accept=".csv,.xlsx"
onchange="handleFileChange(event)" onclick="event.stopPropagation()" />
      <span class="upload-icon">&#128196;</span>
      <p><span class="upload-label">Selecciona un archivo</span> o arrastralo
aqui</p>
      <p style="margin-top:0.4rem; font-size:0.78rem;">Solo .csv y .xlsx</p>
      </div>

      <div class="alert alert-error" id="uploadError"></div>
```

```
      <div class="alert alert-success" id="uploadSuccess"></div>
      </section>
  </div>


  <div class="page" id="page-quienes">
      <div class="static-page">
      <h1>Quiénes somos</h1>
      <p class="lead">
       StockPulse es un proyecto desarrollado por estudiantes del Curso de
Especialización en
       Inteligencia Artificial y Big Data. Nuestro objetivo es construir un
sistema de predicción
      de demanda accesible, fiable y facil de usar para cualquier empresa.
      </p>

      <div class="team-grid">
      <div class="team-card">
      <div class="team-card-avatar">JC</div>
      <h3>Jimmy Chen</h3>
      <p>Business Intelligence</p>
      </div>
      <div class="team-card">
      <div class="team-card-avatar">JF</div>
      <h3>Joaquin Fernandez</h3>
      <p>Platform / Project Manager</p>
      </div>
      <div class="team-card">
      <div class="team-card-avatar">SR</div>
      <h3>Sergio Romero</h3>
      <p>Data Engineer</p>
      </div>
      <div class="team-card">
      <div class="team-card-avatar">DT</div>
      <h3>Deninson Tapia</h3>
      <p>Data Scientist</p>
      </div>
      </div>
      </div>
  </div>


  <div class="page" id="page-contacto">
      <div class="static-page">
      <h1>Contacto</h1>
      <p class="lead">¿Tienes alguna pregunta sobre StockPulse? Escríbenos.</p>
      <div class="contact-form">
      <div class="form-group">
      <label>Nombre</label>
      <input type="text" placeholder="Tu nombre" />
      </div>
      <div class="form-group">
      <label>Email</label>
      <input type="email" placeholder="tu@email.com" />
      </div>
      <div class="form-group">
      <label>Mensaje</label>
      <input type="text" placeholder="Tu mensaje..." />
      </div>
```

```
        <button class="btn btn-primary" style="margin-top:0.5rem;">Enviar
mensaje</button>
        </div>
        </div>
  </div>


  <div class="page" id="page-dashboard">
        <div class="dashboard-layout">

        <aside class="sidebar">
        <div class="sidebar-section">
        <div class="sidebar-title">Navegación</div>
        <a class="sidebar-item active" onclick="showDashboardTab('overview')">
              <span class="sidebar-icon">&#9632;</span> Resumen
        </a>
        <a class="sidebar-item" onclick="showDashboardTab('ventas')">
              <span class="sidebar-icon">&#9650;</span> Ventas
        </a>
        <a class="sidebar-item" onclick="showDashboardTab('predicciones')">
              <span class="sidebar-icon">&#9670;</span> Predicciones
        </a>
        </div>
        <div class="sidebar-section">
        <div class="sidebar-title">Datos</div>
        <a class="sidebar-item" onclick="handleUploadClick()">
              <span class="sidebar-icon">&#8593;</span> Cargar nuevo CSV
        </a>
        </div>
        </aside>

        <main class="dashboard-main">

        <div class="loading" id="dashboardLoading">
        <div class="spinner"></div>
        <span>Procesando datos y entrenando modelo...</span>
        </div>

        <div id="dashboardContent" style="display:none;">

        <div class="dashboard-header">
              <h2>Resumen de ventas</h2>
              <p id="dashboardSubtitle">Datos cargados por <strong
id="dashboardUser"></strong></p>
        </div>

        <div class="kpi-grid">
              <div class="kpi-card">
              <div class="kpi-label">Ventas totales</div>
              <div class="kpi-value" id="kpiVentasTotales">--</div>
              <div class="kpi-sub">Periodo completo</div>
              </div>
              <div class="kpi-card">
              <div class="kpi-label">Unidades vendidas</div>
              <div class="kpi-value" id="kpiUnidades">--</div>
              <div class="kpi-sub">Total de registros</div>
              </div>
              <div class="kpi-card">
              <div class="kpi-label">Productos distintos</div>
```

```
            <div class="kpi-value" id="kpiProductos">--</div>
            <div class="kpi-sub">En el dataset</div>
            </div>
            <div class="kpi-card">
            <div class="kpi-label">Confianza del modelo</div>
            <div class="kpi-value green" id="kpiConfianza">--</div>
            </div>
      </div>

      <div style="margin-bottom:1.25rem; display:flex; align-items:center;
gap:0.75rem;">
            <label style="font-size:0.85rem; font-weight:600;
color:var(--navy);">Producto:</label>
            <select class="product-selector" id="globalProductSelector"
onchange="updateAllCharts()">
            <option value="">Cargando productos...</option>
            </select>
      </div>

      <div class="charts-grid">

            <div class="chart-card">
            <div class="chart-card-header">
            <div>
                  <div class="chart-card-title">Ventas ultimo mes</div>
                  <div class="chart-card-sub">Unidades vendidas por día</div>
            </div>
            </div>
            <canvas id="chartVentasMes"></canvas>
            </div>

            <div class="chart-card">
            <div class="chart-card-header">
            <div>
                  <div class="chart-card-title">Predicción próximo mes</div>
                  <div class="chart-card-sub">Demanda estimada por día</div>
            </div>
            </div>
            <canvas id="chartPrediccion"></canvas>
            </div>

            <div class="chart-card">
            <div class="chart-card-header">
            <div>
                  <div class="chart-card-title">Inversión prevista</div>
                  <div class="chart-card-sub">Coste estimado de reposición</div>
            </div>
            </div>
            <canvas id="chartInversion"></canvas>
            </div>

            <div class="chart-card">
            <div class="chart-card-header">
            <div>
                  <div class="chart-card-title">Seguridad del modelo</div>
                  <div class="chart-card-sub">Porcentaje de confianza por
métrica</div>
            </div>
            </div>
```

```
            <canvas id="chartSeguridad"></canvas>
            </div>

      </div>
      </div>
      </main>
      </div>
  </div>

  <div class="modal-overlay" id="loginModal">
      <div class="modal">
      <div class="modal-header">
      <h2>Iniciar sesión</h2>
      <p>Accede con tu cuenta de StockPulse</p>
      </div>
      <div class="form-group">
      <label>Usuario</label>
      <input type="text" id="loginUsername" placeholder="Usuario"
autocomplete="off" />
      </div>
      <div class="form-group">
      <label>Contraseña</label>
      <input type="password" id="loginPassword" placeholder="Contraseña"
autocomplete="off"
               onkeydown="if(event.key==='Enter') submitLogin()" />
      </div>
      <div class="alert alert-error" id="loginError"></div>
      <div class="modal-footer">
      <button class="btn btn-ghost" onclick="closeLoginModal()">Cancelar</button>
      <button class="btn btn-primary" onclick="submitLogin()"
id="loginBtn">Entrar</button>
      </div>
      </div>
  </div>

  <script src="/static/app.js"></script>
</body>
</html>
```

Archivo: styles.css

Ruta completa: `src/frontend/styles.css`

```
@import
url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700&fami
ly=DM+Mono:wght@400;500&display=swap');

/* Variables*/
:root {
  --bg: #f5f7fa;
  --white: #ffffff;
  --navy: #0d1b2a;
  --navy-mid: #1a2e45;
  --navy-light: #243a52;
  --green: #1db954;
```

```
  --green-dark: #17a347;
  --green-soft: #e8f8ee;
  --text: #0d1b2a;
  --text-muted: #5a7184;
  --border: #dce4ed;
  --shadow-sm: 0 1px 3px rgba(13,27,42,0.08);
  --shadow-md: 0 4px 16px rgba(13,27,42,0.12);
  --shadow-lg: 0 8px 32px rgba(13,27,42,0.16);
  --radius: 12px;
  --radius-sm: 8px;
  --transition: 0.2s ease;
}

/* Reset */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
html { scroll-behavior: smooth; }
body {
  font-family: 'DM Sans', sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.6;
  min-height: 100vh;
}

/* Navbar */
.navbar {
  position: sticky;
  top: 0;
  z-index: 100;
  background: var(--navy);
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 2rem;
  height: 60px;
  box-shadow: var(--shadow-md);
}

.navbar-brand {
  font-weight: 700;
  font-size: 1.2rem;
  color: var(--white);
  letter-spacing: -0.02em;
  display: flex;
  align-items: center;
  gap: 0.5rem;
  text-decoration: none;
}

.navbar-brand span {
  color: var(--green);
}

.navbar-links {
  display: flex;
  align-items: center;
  gap: 0.25rem;
  list-style: none;
}
```

```
.navbar-links a {
  color: rgba(255,255,255,0.75);
  text-decoration: none;
  font-size: 0.9rem;
  font-weight: 500;
  padding: 0.4rem 0.85rem;
  border-radius: var(--radius-sm);
  transition: var(--transition);
}

.navbar-links a:hover,
.navbar-links a.active {
  color: var(--white);
  background: rgba(255,255,255,0.1);
}

.navbar-actions {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

.navbar-user {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  color: var(--white);
  font-size: 0.9rem;
  font-weight: 500;
}

.navbar-avatar {
  width: 32px;
  height: 32px;
  border-radius: 50%;
  background: var(--green);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 0.8rem;
  font-weight: 700;
  color: var(--white);
}

/* Botones */
.btn {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  padding: 0.5rem 1.1rem;
  border-radius: var(--radius-sm);
  font-family: 'DM Sans', sans-serif;
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  border: none;
  transition: var(--transition);
  text-decoration: none;
```

```
  white-space: nowrap;
}

.btn-primary {
  background: var(--green);
  color: var(--white);
}
.btn-primary:hover { background: var(--green-dark); transform: translateY(-1px);
box-shadow: var(--shadow-sm); }

.btn-outline {
  background: transparent;
  color: var(--white);
  border: 1.5px solid rgba(255,255,255,0.35);
}
.btn-outline:hover { background: rgba(255,255,255,0.1); border-color:
rgba(255,255,255,0.6); }

.btn-ghost {
  background: transparent;
  color: var(--text-muted);
  border: 1.5px solid var(--border);
}
.btn-ghost:hover { background: var(--bg); color: var(--text); border-color:
var(--navy-light); }

.btn-danger {
  background: transparent;
  color: #e74c3c;
  border: 1.5px solid rgba(231,76,60,0.3);
}
.btn-danger:hover { background: rgba(231,76,60,0.08); }

.btn-lg {
  padding: 0.8rem 2rem;
  font-size: 1rem;
  border-radius: var(--radius);
}

.btn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
  transform: none !important;
}

/* Pages */
.page { display: none; min-height: calc(100vh - 60px); }
.page.active { display: block; }

/* Hero (Inicio) */
.hero {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  text-align: center;
  padding: 6rem 2rem 4rem;
  min-height: calc(100vh - 60px);
```

```
  background: linear-gradient(160deg, var(--navy) 0%, var(--navy-mid) 50%, #0f2535
100%);
  position: relative;
  overflow: hidden;
}

.hero::before {
  content: '';
  position: absolute;
  top: -50%;
  left: -50%;
  width: 200%;
  height: 200%;
  background: radial-gradient(circle at 60% 40%, rgba(29,185,84,0.08) 0%,
transparent 50%),
              radial-gradient(circle at 20% 80%, rgba(29,185,84,0.05) 0%,
transparent 40%);
  pointer-events: none;
}

.hero-title {
  font-size: 8rem;
  font-weight: 800;
  color: var(--green);
  margin-bottom: 0.75rem;
  letter-spacing: -0.02em;
}

.hero-badge {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  background: rgba(29,185,84,0.15);
  border: 1px solid rgba(29,185,84,0.3);
  color: var(--green);
  font-size: 0.8rem;
  font-weight: 600;
  padding: 0.3rem 0.9rem;
  border-radius: 100px;
  margin-bottom: 1.5rem;
  letter-spacing: 0.04em;
  text-transform: uppercase;
}

.hero h1 {
  font-size: clamp(2.5rem, 6vw, 4.5rem);
  font-weight: 700;
  color: var(--white);
  line-height: 1.1;
  letter-spacing: -0.03em;
  margin-bottom: 1.25rem;
}

.hero h1 span { color: var(--green); }

.hero p {
  font-size: 1.15rem;
  color: rgba(255,255,255,0.65);
  max-width: 520px;
```

```
  margin-bottom: 2.5rem;
  line-height: 1.7;
}

.hero-actions {
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;
  justify-content: center;
}

/* Upload area */
.upload-area {
  margin-top: 3rem;
  border: 2px dashed rgba(255,255,255,0.2);
  border-radius: var(--radius);
  padding: 2rem 3rem;
  cursor: pointer;
  transition: var(--transition);
  max-width: 480px;
  width: 100%;
  text-align: center;
  position: relative;
}

.upload-area:hover {
  border-color: var(--green);
  background: rgba(29,185,84,0.05);
}

.upload-area input[type="file"] {
  position: absolute;
  inset: 0;
  opacity: 0;
  cursor: pointer;
  width: 100%;
  height: 100%;
}

.upload-icon {
  font-size: 2rem;
  margin-bottom: 0.75rem;
  display: block;
}

.upload-area p {
  color: rgba(255,255,255,0.6);
  font-size: 0.9rem;
  margin: 0;
}

.upload-area .upload-label {
  color: var(--green);
  font-weight: 600;
}

/* Error/Success */
.alert {
  display: none;
```

```
  padding: 0.75rem 1rem;
  border-radius: var(--radius-sm);
  font-size: 0.875rem;
  font-weight: 500;
  margin-top: 1rem;
  max-width: 480px;
  width: 100%;
  text-align: left;
}

.alert.show { display: flex; align-items: center; gap: 0.5rem; }
.alert-error { background: rgba(231,76,60,0.1); border: 1px solid
rgba(231,76,60,0.3); color: #e74c3c; }
.alert-success { background: var(--green-soft); border: 1px solid
rgba(29,185,84,0.3); color: var(--green-dark); }

/* Login Modal */
.modal-overlay {
  display: none;
  position: fixed;
  inset: 0;
  background: rgba(13,27,42,0.7);
  backdrop-filter: blur(4px);
  z-index: 200;
  align-items: center;
  justify-content: center;
}

.modal-overlay.show { display: flex; }

.modal {
  background: var(--white);
  border-radius: var(--radius);
  padding: 2.5rem;
  width: 100%;
  max-width: 400px;
  box-shadow: var(--shadow-lg);
  animation: modalIn 0.25s ease;
}

@keyframes modalIn {
  from { opacity: 0; transform: translateY(-16px) scale(0.97); }
  to { opacity: 1; transform: translateY(0) scale(1); }
}

.modal-header {
  margin-bottom: 1.75rem;
}

.modal-header h2 {
  font-size: 1.4rem;
  font-weight: 700;
  color: var(--navy);
  letter-spacing: -0.02em;
}

.modal-header p {
  color: var(--text-muted);
  font-size: 0.875rem;
```

```
  margin-top: 0.25rem;
}

.form-group {
  margin-bottom: 1.1rem;
}

.form-group label {
  display: block;
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--navy);
  margin-bottom: 0.4rem;
}

.form-group input {
  width: 100%;
  padding: 0.65rem 0.9rem;
  border: 1.5px solid var(--border);
  border-radius: var(--radius-sm);
  font-family: 'DM Sans', sans-serif;
  font-size: 0.9rem;
  color: var(--text);
  background: var(--bg);
  transition: var(--transition);
  outline: none;
}

.form-group input:focus {
  border-color: var(--navy);
  background: var(--white);
  box-shadow: 0 0 0 3px rgba(13,27,42,0.07);
}

.modal-footer {
  display: flex;
  justify-content: flex-end;
  gap: 0.75rem;
  margin-top: 1.5rem;
}

/* Dashboard */
.dashboard-layout {
  display: grid;
  grid-template-columns: 240px 1fr;
  min-height: calc(100vh - 60px);
}

.sidebar {
  background: var(--white);
  border-right: 1px solid var(--border);
  padding: 1.5rem 1rem;
}

.sidebar-section {
  margin-bottom: 2rem;
}

.sidebar-title {
```

```
  font-size: 0.7rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: var(--text-muted);
  padding: 0 0.5rem;
  margin-bottom: 0.5rem;
}

.sidebar-item {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  padding: 0.55rem 0.75rem;
  border-radius: var(--radius-sm);
  color: var(--text-muted);
  font-size: 0.875rem;
  font-weight: 500;
  cursor: pointer;
  transition: var(--transition);
  text-decoration: none;
}

.sidebar-item:hover { background: var(--bg); color: var(--text); }
.sidebar-item.active { background: var(--green-soft); color: var(--green-dark);
font-weight: 600; }

.sidebar-icon { font-size: 1rem; width: 20px; text-align: center; }

.dashboard-main {
  padding: 2rem;
  overflow-y: auto;
}

.dashboard-header {
  margin-bottom: 1.75rem;
}

.dashboard-header h2 {
  font-size: 1.5rem;
  font-weight: 700;
  color: var(--navy);
  letter-spacing: -0.02em;
}

.dashboard-header p {
  color: var(--text-muted);
  font-size: 0.875rem;
  margin-top: 0.2rem;
}

/* KPI Cards */
.kpi-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1rem;
  margin-bottom: 1.75rem;
}
```

```
.kpi-card {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 1.25rem 1.5rem;
  box-shadow: var(--shadow-sm);
  transition: var(--transition);
}

.kpi-card:hover { box-shadow: var(--shadow-md); transform: translateY(-2px); }

.kpi-label {
  font-size: 0.78rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  color: var(--text-muted);
  margin-bottom: 0.5rem;
}

.kpi-value {
  font-size: 1.75rem;
  font-weight: 700;
  color: var(--navy);
  letter-spacing: -0.03em;
  line-height: 1;
  font-family: 'DM Mono', monospace;
}

.kpi-value.green { color: var(--green-dark); }

.kpi-sub {
  font-size: 0.78rem;
  color: var(--text-muted);
  margin-top: 0.4rem;
}

/* Charts grid */
.charts-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 1.25rem;
}

.chart-card {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 1.5rem;
  box-shadow: var(--shadow-sm);
}

.chart-card-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 1.25rem;
}
```

```
.chart-card-title {
  font-size: 0.9rem;
  font-weight: 700;
  color: var(--navy);
}

.chart-card-sub {
  font-size: 0.78rem;
  color: var(--text-muted);
  margin-top: 0.15rem;
}

.product-selector {
  padding: 0.4rem 0.7rem;
  border: 1.5px solid var(--border);
  border-radius: var(--radius-sm);
  font-family: 'DM Sans', sans-serif;
  font-size: 0.8rem;
  color: var(--text);
  background: var(--bg);
  outline: none;
  cursor: pointer;
  transition: var(--transition);
}

.product-selector:focus { border-color: var(--navy); }

.chart-placeholder {
  height: 200px;
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--text-muted);
  font-size: 0.85rem;
  background: var(--bg);
  border-radius: var(--radius-sm);
}

canvas { max-height: 220px; }

/* Loading spinner */
.loading {
  display: none;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 1rem;
  padding: 3rem;
  color: var(--text-muted);
  font-size: 0.9rem;
}

.loading.show { display: flex; }

.spinner {
  width: 36px;
  height: 36px;
  border: 3px solid var(--border);
  border-top-color: var(--green);
```

```
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}

@keyframes spin { to { transform: rotate(360deg); } }

/* Static page */
.static-page {
  max-width: 760px;
  margin: 0 auto;
  padding: 4rem 2rem;
}

.static-page h1 {
  font-size: 2rem;
  font-weight: 700;
  color: var(--navy);
  letter-spacing: -0.03em;
  margin-bottom: 0.5rem;
}

.static-page .lead {
  color: var(--text-muted);
  font-size: 1.05rem;
  margin-bottom: 2.5rem;
  line-height: 1.7;
}

.team-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.25rem;
  margin-top: 2rem;
}

.team-card {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 1.5rem 1rem;
  text-align: center;
  box-shadow: var(--shadow-sm);
}

.team-card-avatar {
  width: 52px;
  height: 52px;
  border-radius: 50%;
  background: var(--navy);
  color: var(--white);
  font-size: 1.1rem;
  font-weight: 700;
  display: flex;
  align-items: center;
  justify-content: center;
  margin: 0 auto 0.75rem;
}

.team-card h3 {
```

```
  font-size: 0.9rem;
  font-weight: 700;
  color: var(--navy);
}

.team-card p {
  font-size: 0.78rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.contact-form {
  background: var(--white);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 2rem;
  box-shadow: var(--shadow-sm);
  margin-top: 1.5rem;
}

/* Responsive */
@media (max-width: 768px) {
  .charts-grid { grid-template-columns: 1fr; }
  .dashboard-layout { grid-template-columns: 1fr; }
  .sidebar { display: none; }
  .hero h1 { font-size: 2rem; }
}
```

Archivo: app.js

Ruta completa: `src/frontend/app.js`

```
const API_BASE = 'http://localhost:8000';

// Estado de sesión (solo en memoria, sin localStorage)
const session = {
  token: null,
  username: null,
  isLoggedIn: function () { return !!this.token; }
};

const charts = {};

// NAVEGACIÓN

function navigateTo(page) {
  document.querySelectorAll('.page').forEach(p => p.classList.remove('active'));
  document.querySelectorAll('[data-page]').forEach(a =>
a.classList.remove('active'));

  const target = document.getElementById('page-' + page);
```

```
  if (target) target.classList.add('active');

  const link = document.querySelector(`[data-page="${page}"]`);
  if (link) link.classList.add('active');

  if (page === 'dashboard') {
       if (!session.isLoggedIn()) { openLoginModal(); return; }
    loadDashboard();
  }

  return false;
}

// AUTENTICACIÓN

function openLoginModal() {
  document.getElementById('loginModal').classList.add('show');
  document.getElementById('loginUsername').focus();
  hideAlert('loginError');
}

function closeLoginModal() {
  document.getElementById('loginModal').classList.remove('show');
  document.getElementById('loginUsername').value = '';
  document.getElementById('loginPassword').value = '';
}

async function submitLogin() {
  const username = document.getElementById('loginUsername').value.trim();
  const password = document.getElementById('loginPassword').value;
  const btn = document.getElementById('loginBtn');

  if (!username || !password) {
    showAlert('loginError', 'Rellena todos los campos.');
    return;
  }

  btn.disabled = true;
  btn.textContent = 'Entrando...';

  try {
       const res = await fetch(`${API_BASE}/api/auth/login`, {
       method: 'POST',
       headers: { 'Content-Type': 'application/json' },
       body: JSON.stringify({ username, password })
       });

       const data = await res.json();

       if (!res.ok) {
      showAlert('loginError', data.detail || 'Usuario o contraseña incorrectos.');
      return;
       }

       // Guardar sesión en memoria
    session.token = data.token;
    session.username = data.username || username;

    closeLoginModal();
```

```
    updateNavbar();
    navigateTo('dashboard');

  } catch (err) {
    showAlert('loginError', 'No se pudo conectar con el servidor.');
  } finally {
    btn.disabled = false;
    btn.textContent = 'Entrar';
  }
}

async function logout() {
  if (session.token) {
      try {
      await fetch(`${API_BASE}/api/auth/logout`, {
      method: 'POST',
      headers: authHeaders()
      });
      } catch (_) {}
  }

  // Limpiar sesión de memoria
  session.token = null;
  session.username = null;

  // Destruir gráficos
  Object.values(charts).forEach(c => { if (c) c.destroy(); });
  Object.keys(charts).forEach(k => delete charts[k]);

  updateNavbar();
  navigateTo('inicio');

  document.getElementById('uploadArea').style.display = 'none';
}

function updateNavbar() {
  const guest = document.getElementById('navGuest');
  const user = document.getElementById('navUser');
  const dashItem = document.getElementById('navDashboardItem');
  const navUsername = document.getElementById('navUsername');
  const navAvatar = document.getElementById('navAvatar');
  const uploadArea = document.getElementById('uploadArea');

  if (session.isLoggedIn()) {
    guest.style.display = 'none';
    user.style.display = 'flex';
    dashItem.style.display = 'block';
    navUsername.textContent = session.username;
    navAvatar.textContent = session.username.charAt(0).toUpperCase();
    uploadArea.style.display = 'block';
  } else {
    guest.style.display = 'block';
    user.style.display = 'none';
    dashItem.style.display = 'none';
    uploadArea.style.display = 'none';
  }
}

function authHeaders() {
```

```
  return {
       'Authorization': `Bearer ${session.token}`,
       'Content-Type': 'application/json'
  };
}

// CARGA DE ARCHIVOS

function handleUploadClick() {
  if (!session.isLoggedIn()) {
    openLoginModal();
    return;
  }
  triggerFileInput();
}

function triggerFileInput() {
  document.getElementById('fileInput').click();
}

async function handleFileChange(event) {
  const file = event.target.files[0];
  if (!file) return;

  hideAlert('uploadError');
  hideAlert('uploadSuccess');

  const ext = file.name.split('.').pop().toLowerCase();
  if (!['csv', 'xlsx'].includes(ext)) {
    showAlert('uploadError', 'Formato no válido. Solo se permiten archivos .csv y
.xlsx');
    event.target.value = '';
    return;
  }

  if (file.size > 50 * 1024 * 1024) {
    showAlert('uploadError', 'El archivo es demasiado grande. Máximo 50MB.');
    event.target.value = '';
    return;
  }

  showAlert('uploadSuccess', `Archivo "${file.name}" seleccionado. Procesando...`);

  const formData = new FormData();
  formData.append('file', file);
  formData.append('username', session.username);

  try {
       const res = await fetch(`${API_BASE}/api/upload`, {
       method: 'POST',
       headers: { 'Authorization': `Bearer ${session.token}` },
       body: formData
       });

       const data = await res.json();

       if (!res.ok) {
      showAlert('uploadError', data.detail || 'Error al procesar el archivo.');
      return;
```

```
      }

    showAlert('uploadSuccess', 'Archivo procesado correctamente. Redirigiendo al
dashboard...');

    setTimeout(() => {
      hideAlert('uploadSuccess');
      navigateTo('dashboard');
       }, 1500);

  } catch (err) {
    showAlert('uploadError', 'No se pudo conectar con el servidor.');
  } finally {
    event.target.value = '';
  }
}

// DASHBOARD

async function loadDashboard() {
  const loading = document.getElementById('dashboardLoading');
  const content = document.getElementById('dashboardContent');

  loading.classList.add('show');
  content.style.display = 'none';

  try {
       // Cargar KPIs y lista de productos en paralelo
       const [kpisRes, productosRes] = await Promise.all([
       fetch(`${API_BASE}/api/dashboard/kpis`, { headers: authHeaders() }),
       fetch(`${API_BASE}/api/dashboard/productos`, { headers: authHeaders() }),
       ]);

       if (!kpisRes.ok || !productosRes.ok) {
      console.error('Error cargando datos del dashboard.');
      return;
       }

       const kpis = await kpisRes.json();
       const productos = await productosRes.json();

       // Actualizar KPIs
    document.getElementById('kpiVentasTotales').textContent =
       '£' + kpis.ventas_totales.toLocaleString('es-ES', { minimumFractionDigits: 2
});
    document.getElementById('kpiUnidades').textContent =
      kpis.unidades_totales.toLocaleString('es-ES');
    document.getElementById('kpiProductos').textContent =
      kpis.productos_distintos.toLocaleString('es-ES');

       // Mostrar usuario en cabecera del dashboard
    document.getElementById('dashboardUser').textContent = session.username;

       // Rellenar selector de productos
       const selector = document.getElementById('globalProductSelector');
    selector.innerHTML = '';
    productos.forEach(p => {
       const opt = document.createElement('option');
      opt.value = p.id_producto;
```

```
      opt.textContent = `${p.id_producto} — ${p.nombre.substring(0, 40)}`;
      selector.appendChild(opt);
       });

       // Cargar gráficos para el primer producto de la lista
       if (productos.length > 0) {
       await updateAllCharts();
       }

    loading.classList.remove('show');
    content.style.display = 'block';

  } catch (err) {
    console.error('Error en loadDashboard:', err);
    loading.classList.remove('show');
  }
}

async function updateAllCharts() {
  const idProducto = document.getElementById('globalProductSelector').value;
  if (!idProducto) return;

  // Cargar ventas, predicciones e inversión en paralelo
  const [ventasRes, predRes, invRes] = await Promise.all([
       fetch(`${API_BASE}/api/dashboard/ventas/${idProducto}`, { headers:
authHeaders() }),
       fetch(`${API_BASE}/api/dashboard/predicciones/${idProducto}`, { headers:
authHeaders() }),
       fetch(`${API_BASE}/api/dashboard/inversion/${idProducto}`, { headers:
authHeaders() }),
  ]);

  const ventas      = await ventasRes.json();
  const predicciones = await predRes.json();
  const inversion   = await invRes.json();

  // Actualizar KPI de confianza del modelo
  const kpiConf = document.getElementById('kpiConfianza');
  if (predicciones.confianza !== null && predicciones.confianza !== undefined) {
    kpiConf.textContent = (predicciones.confianza * 100).toFixed(1) + '%';
  } else {
    kpiConf.textContent = 'Sin datos';
  }

  renderChart('chartVentasMes', {
       labels: ventas.fechas,
       datasets: [{
       label: 'Unidades vendidas',
       data: ventas.unidades,
      borderColor: '#1db954',
      backgroundColor: 'rgba(29,185,84,0.1)',
      borderWidth: 2,
       fill: true,
       tension: 0.3,
      pointRadius: 2,
       }]
  });

  renderChart('chartPrediccion', {
```

```
        labels: predicciones.fechas,
        datasets: [{
        label: 'Demanda estimada',
        data: predicciones.predichas,
       borderColor: '#2980b9',
       backgroundColor: 'rgba(41,128,185,0.1)',
       borderWidth: 2,
        fill: true,
        tension: 0.3,
       pointRadius: 2,
        }]
  });

  renderChart('chartInversion', {
        labels: inversion.fechas,
        datasets: [{
        label: 'Coste estimado (£)',
        data: inversion.inversion,
       borderColor: '#e67e22',
       backgroundColor: 'rgba(230,126,34,0.1)',
       borderWidth: 2,
        fill: true,
        tension: 0.3,
       pointRadius: 2,
        }]
  });

  // Gráfico de seguridad del modelo (dona con métricas de confianza)
  const conf = predicciones.confianza ? predicciones.confianza * 100 : 0;
  renderDoughnut('chartSeguridad', conf);
}

function renderChart(canvasId, data) {
  // Destruir gráfico anterior si existe
  if (charts[canvasId]) {
        charts[canvasId].destroy();
  }

  const ctx = document.getElementById(canvasId).getContext('2d');
  charts[canvasId] = new Chart(ctx, {
        type: 'line',
        data: data,
        options: {
        responsive: true,
        plugins: {
        legend: { display: false },
        },
        scales: {
        x: {
        ticks: { maxTicksLimit: 8, font: { size: 10 } },
        grid: { display: false },
        },
        y: {
        ticks: { font: { size: 10 } },
        grid: { color: 'rgba(0,0,0,0.05)' },
        }
        }
        }
  });
```

```
}

function renderDoughnut(canvasId, confianzaPct) {
  if (charts[canvasId]) {
       charts[canvasId].destroy();
  }

  const resto = 100 - confianzaPct;
  const ctx = document.getElementById(canvasId).getContext('2d');
  charts[canvasId] = new Chart(ctx, {
       type: 'doughnut',
       data: {
       labels: ['Confianza', 'Margen de error'],
       datasets: [{
       data: [confianzaPct, resto],
        backgroundColor: ['#1db954', '#dce4ed'],
        borderWidth: 0,
       }]
       },
       options: {
       responsive: true,
       cutout: '70%',
       plugins: {
       legend: { position: 'bottom', labels: { font: { size: 11 } } },
       }
       }
  });
}

// SIDEBAR TABS

function showDashboardTab(tab) {
  document.querySelectorAll('.sidebar-item').forEach(i =>
i.classList.remove('active'));
  event.currentTarget.classList.add('active');
}

// ALERTAS

function showAlert(id, message) {
  const el = document.getElementById(id);
  el.textContent = message;
  el.classList.add('show');
}

function hideAlert(id) {
  const el = document.getElementById(id);
  if (el) el.classList.remove('show');
}

updateNavbar();
```

Archivo: features.py

Ruta completa: `src/models/features.py`

```
import numpy as np


def build_features(df):
      """
      Aqui creamos todas las features que le damos al modelo
      para que aprenda los patrones de ventas.
      """
    df = df.sort_values("fecha")

      # -- Lags: ventas de dias anteriores para que el modelo sepa que paso antes
--
      for lag in [1, 3, 7, 14, 21, 30]:
       df[f"lag_{lag}"] = df["ventas"].shift(lag)

      # -- Medias moviles: para suavizar y ver tendencias --
      for win in [3, 7, 14, 21, 30]:
       df[f"media_{win}"] = df["ventas"].rolling(win).mean()

      # -- Desviacion estandar movil: mide cuanto varian las ventas --
      for win in [7, 14, 30]:
       df[f"std_{win}"] = df["ventas"].rolling(win).std()

      # -- Cosas del calendario --
    df["dia_semana"]  = df["fecha"].dt.dayofweek
    df["dia_mes"]     = df["fecha"].dt.day
    df["mes"]       = df["fecha"].dt.month
    df["semana_anio"] = df["fecha"].dt.isocalendar().week.astype(int)
    df["trimestre"]   = df["fecha"].dt.quarter
    df["es_finde"]    = df["dia_semana"].isin([5, 6]).astype(int)

      # -- Tendencia: cuantos dias han pasado desde el inicio --
    df["tendencia"] = (df["fecha"] - df["fecha"].min()).dt.days

      # Quitamos las filas que tienen NaN por los shifts y rollings
    df = df.dropna()

      return df


def tratar_outliers(df, columna="ventas"):
      """
    Los outliers nos fastidian el modelo, asi que los capeamos.
    Usamos el metodo IQR: si un valor se pasa mucho del rango normal,
      lo recortamos al limite. Asi no perdemos filas pero tampoco
    nos distorsionan las predicciones.
      """
      q1 = df[columna].quantile(0.25)
      q3 = df[columna].quantile(0.75)
    iqr = q3 - q1

    limite_bajo = q1 - 1.5 * iqr
    limite_alto = q3 + 1.5 * iqr

      antes = len(df[(df[columna] < limite_bajo) | (df[columna] > limite_alto)])
    print(f"  Outliers detectados: {antes}")
    print(f"  Rango permitido: [{limite_bajo:.0f}, {limite_alto:.0f}]")

      # Capeamos en vez de eliminar (clip), para no perder dias
```

```
    df[columna] = df[columna].clip(lower=limite_bajo, upper=limite_alto)

       return df


def aplicar_log(df, columna="ventas"):
       """
    Aplicamos log1p a las ventas para que la distribucion sea mas normal.
       log1p es log(1+x) para evitar problemas con el 0.
       Esto ayuda a que los modelos entrenen mejor cuando los datos estan
    muy sesgados (muchos valores bajos y pocos muy altos).
       """
    print(f"  Skewness antes del log: {df[columna].skew():.2f}")
    df[columna] = np.log1p(df[columna])
    print(f"  Skewness despues del log: {df[columna].skew():.2f}")
       return df
```

Archivo: train_model.py

Ruta completa: `src/models/train_model.py`

```
"""
train_model.py
Proyecto StockPulse - Grupo 5
Autor: Deninson Tapia

Este es el script del modelo definitivo.
Despues de comparar Ridge, Random Forest y Gradient Boosting
en compare_models.py, Ridge fue el ganador asi que aqui
lo entrenamos, guardamos el modelo y exportamos las predicciones
para que Jimmy las use en el dashboard.

Modelo elegido: Ridge (alpha=10.0)
Motivo: mejor RMSE, MAE y R2 en la comparacion. Ademas con
un dataset tan pequeno (~220 dias de train) un modelo simple
generaliza mejor que los ensemble.
"""

import os
import sys

import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
import joblib

sys.path.append(os.path.join(os.path.dirname(__file__), "..", ".."))
from src.models.features import build_features, tratar_outliers, aplicar_log
```

```python
# --- Rutas ---
DEFAULT_INPUT = os.path.join("data", "processed", "ventas_limpias.csv")
RESULTS_DIR   = os.path.join("data", "results")
MODEL_DIR     = os.path.join("src", "models", "saved")

# --- El modelo ganador ---
ALPHA = 10.0
TEST_RATIO = 0.20

FEATURES = [
       "lag_1", "lag_3", "lag_7", "lag_14", "lag_21", "lag_30",
       "media_3", "media_7", "media_14", "media_21", "media_30",
       "std_7", "std_14", "std_30",
       "dia_semana", "dia_mes", "mes", "semana_anio", "trimestre",
       "es_finde", "tendencia",
]
TARGET = "ventas"


def load_and_aggregate(filepath):
       """Cargamos el csv y agrupamos por dia."""
    print(f"Cargando dataset: {filepath}")
    df = pd.read_csv(filepath, parse_dates=["fecha_venta"], low_memory=False)

       daily = (
        df.groupby("fecha_venta")["unidades_vendidas"]
        .sum()
       .reset_index()
        .rename(columns={"fecha_venta": "fecha", "unidades_vendidas": "ventas"})
       .sort_values("fecha")
       )

    print(f"Serie temporal: {len(daily)} dias "
       f"({daily['fecha'].min().date()} -> {daily['fecha'].max().date()})")
       return daily


def prepare_data(df):
       """
    Mismo preprocesamiento que en compare_models:
       outliers -> log -> features -> split temporal
       """
    print("\n--- Preprocesamiento ---")

    print("Tratando outliers:")
    df = tratar_outliers(df, "ventas")

    print("Aplicando transformacion log:")
    df = aplicar_log(df, "ventas")

    df = build_features(df)

       # Split temporal
    n_test  = int(len(df) * TEST_RATIO)
    n_train = len(df) - n_test

       train = df.iloc[:n_train]
    test  = df.iloc[n_train:]
```

```
    print(f"Train: {len(train)} dias | Test: {len(test)} dias")

    return train, test


def train_and_evaluate(train, test):
    """Entrenamos Ridge y evaluamos."""

    X_train, y_train = train[FEATURES], train[TARGET]
    X_test,  y_test  = test[FEATURES],  test[TARGET]

    # Escalamos features (Ridge lo necesita por la regularizacion)
    scaler = StandardScaler()
    X_train_sc = scaler.fit_transform(X_train)
    X_test_sc  = scaler.transform(X_test)

    # Entrenamos
    print(f"\nEntrenando Ridge (alpha={ALPHA})...")
    modelo = Ridge(alpha=ALPHA)
    modelo.fit(X_train_sc, y_train)

    # Predicciones en escala real (deshacemos el log)
    y_pred_log  = modelo.predict(X_test_sc)
    y_pred_real = np.expm1(np.clip(y_pred_log, 0, None))
    y_test_real = np.expm1(y_test)

    # Metricas
    rmse = np.sqrt(mean_squared_error(y_test_real, y_pred_real))
    mae  = mean_absolute_error(y_test_real, y_pred_real)
    r2   = r2_score(y_test_real, y_pred_real)

    print(f"\n--- Resultados modelo definitivo ---")
    print(f"  RMSE: {rmse:.2f}")
    print(f"  MAE:  {mae:.2f}")
    print(f"  R2:   {r2:.4f}")

    return modelo, scaler, y_test_real, y_pred_real, test["fecha"]


def save_model(modelo, scaler):
    """Guardamos el modelo y el scaler para poder usarlos despues."""
    os.makedirs(MODEL_DIR, exist_ok=True)

    model_path  = os.path.join(MODEL_DIR, "ridge_model.pkl")
    scaler_path = os.path.join(MODEL_DIR, "scaler.pkl")

    joblib.dump(modelo, model_path)
    joblib.dump(scaler, scaler_path)

    print(f"\nModelo guardado en: {model_path}")
    print(f"Scaler guardado en: {scaler_path}")


def save_predictions(fechas, y_real, y_pred):
    """
    Exportamos las predicciones a CSV para que se puedan usar
    en el dashboard o en la memoria del proyecto.
    """
```

```
    os.makedirs(RESULTS_DIR, exist_ok=True)

    df_pred = pd.DataFrame({
       "fecha":           fechas.values,
       "ventas_reales":   y_real.values,
       "ventas_predichas": y_pred,
       })

    pred_path = os.path.join(RESULTS_DIR, "predicciones_ridge.csv")
    df_pred.to_csv(pred_path, index=False)
    print(f"Predicciones guardadas en: {pred_path}")


def plot_results(fechas, y_real, y_pred):
       """Grafica del modelo definitivo: real vs prediccion."""
       fig, ax = plt.subplots(figsize=(13, 5))

    ax.plot(fechas, y_real.values, label="Real", color="black", linewidth=1.5)
    ax.plot(fechas, y_pred, label="Ridge (prediccion)", color="#e74c3c",
            linestyle="--", linewidth=2)

    ax.set_title("Modelo definitivo (Ridge) - Prediccion vs Ventas reales",
fontsize=13)
    ax.set_xlabel("Fecha")
    ax.set_ylabel("Unidades vendidas")
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
    ax.xaxis.set_major_locator(mdates.MonthLocator())
    plt.xticks(rotation=30)
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.tight_layout()

    plot_path = os.path.join(RESULTS_DIR, "prediccion_ridge.png")
    plt.savefig(plot_path, dpi=150)
    print(f"Grafica guardada en: {plot_path}")
    plt.close()


def run_training(filepath):
       """
    Ejecuta el pipeline de entrenamiento completo y devuelve el DataFrame
       de predicciones (fecha, ventas_reales, ventas_predichas) para que el
    orquestador lo pueda insertar en la base de datos sin releer el CSV.

    Efectos secundarios: guarda modelo, scaler, CSV de predicciones y gráfica.
       """
       # Lo llama run_pipeline.py justo después de cargar los datos en BD,
       # para encadenar "datos limpios -> modelo entrenado -> predicciones".
    df_daily = load_and_aggregate(filepath)
       train, test = prepare_data(df_daily)

    modelo, scaler, y_real, y_pred, fechas = train_and_evaluate(train, test)

    save_model(modelo, scaler)
    save_predictions(fechas, y_real, y_pred)
    plot_results(fechas, y_real, y_pred)

       return pd.DataFrame({
       "fecha":            pd.to_datetime(fechas.values).date,
```

```
        "ventas_reales":    y_real.values,
        "ventas_predichas": y_pred,
        })


def main(filepath):
    run_training(filepath)
    print("\nTodo listo!")


if __name__ == "__main__":
        import argparse
        parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=str, default=DEFAULT_INPUT, help="Ruta al
CSV limpio")
    args = parser.parse_args()
        main(args.input)
```

Archivo: __init__.py

Ruta completa: `src/pipeline/__init__.py`

Archivo: ingest.py

Ruta completa: `src/pipeline/ingest.py`

```
"""
ingest.py — Capa de ingesta de datos
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Responsabilidad:
        Carga el dataset original (CSV o XLSX) y lo devuelve como DataFrame de
Pandas.
        Valida que las columnas necesarias existan antes de continuar el pipeline.
"""

import pandas as pd
import os
```

```
# Columnas mínimas que debe tener el dataset de entrada
REQUIRED_COLUMNS = {
       "InvoiceNo",
       "StockCode",
       "Description",
       "Quantity",
       "InvoiceDate",
       "UnitPrice",
}


def load_dataset(filepath: str) -> pd.DataFrame:
       """
       Carga un archivo CSV o XLSX y lo devuelve como DataFrame.

    Parámetros:
        filepath (str): Ruta absoluta o relativa al archivo de datos.

    Retorna:
        pd.DataFrame: Dataset cargado sin procesar.

    Excepciones:
        FileNotFoundError: Si el archivo no existe en la ruta indicada.
        ValueError: Si la extensión no es CSV ni XLSX.
        ValueError: Si faltan columnas obligatorias.
       """

       # Verificar que el archivo existe
       if not os.path.exists(filepath):
       raise FileNotFoundError(f"No se encontró el archivo: {filepath}")

       extension = os.path.splitext(filepath)[1].lower()

       # Cargar según el formato del archivo
       if extension == ".csv":
       print(f"[INGESTA] Cargando CSV: {filepath}")
        df = pd.read_csv(filepath, encoding="utf-8", low_memory=False)

    elif extension == ".xlsx":
       print(f"[INGESTA] Cargando XLSX: {filepath}")
        df = pd.read_excel(filepath, engine="openpyxl")

       else:
       raise ValueError(f"Formato no soportado: '{extension}'. Usa CSV o XLSX.")

       print(f"[INGESTA] Dataset cargado: {df.shape[0]} filas, {df.shape[1]}
columnas.")

       # Validar que las columnas necesarias existen
       _validate_columns(df)

       return df


def _validate_columns(df: pd.DataFrame) -> None:
       """
    Comprueba que el DataFrame contiene todas las columnas requeridas.
```

```
    Parámetros:
        df (pd.DataFrame): DataFrame a validar.

    Excepciones:
        ValueError: Lista las columnas que faltan si no están todas presentes.
       """
    columnas_presentes = set(df.columns)
    columnas_faltantes = REQUIRED_COLUMNS - columnas_presentes

       if columnas_faltantes:
       raise ValueError(
              f"[INGESTA] Faltan columnas obligatorias en el dataset:
{columnas_faltantes}"
       )

       print(f"[INGESTA] Validación de columnas OK.")
```

Archivo: clean.py

Ruta completa: `src/pipeline/clean.py`

```
"""
clean.py — Capa de limpieza y normalización de datos
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Responsabilidad:
    Recibe el DataFrame en bruto de la capa de ingesta y aplica:
       1. Eliminación de cancelaciones (InvoiceNo que empieza por 'C')
       2. Eliminación de filas con Quantity <= 0 (devoluciones)
       3. Eliminación de filas con UnitPrice <= 0
       4. Eliminación de duplicados exactos
       5. Gestión de nulos (Description y CustomerID)
       6. Normalización del formato de fechas a YYYY-MM-DD
       7. Normalización de texto (Description, StockCode)
       8. Cálculo de total_venta = Quantity * UnitPrice
       9. Renombrado de columnas al esquema interno del proyecto

       Al final devuelve dos DataFrames listos para insertar:
       - df_productos: columnas del modelo Productos
       - df_ventas:  columnas del modelo Ventas
"""

import pandas as pd


def clean_and_normalize(df: pd.DataFrame) -> pd.DataFrame:
       """
    Aplica todas las operaciones de limpieza sobre el DataFrame bruto.

    Parámetros:
        df (pd.DataFrame): DataFrame original cargado por ingest.py.

    Retorna:
        pd.DataFrame: DataFrame limpio y normalizado.
```

```
   """

filas_inicio = len(df)
   print(f"[LIMPIEZA] Iniciando con {filas_inicio} filas.")

   # --- 1. Eliminar cancelaciones (InvoiceNo empieza por 'C') ---
   # Las facturas canceladas tienen 'C' como prefijo en el número de factura
df = df[~df["InvoiceNo"].astype(str).str.startswith("C")]
   print(f"[LIMPIEZA] Tras eliminar cancelaciones: {len(df)} filas.")

   # --- 2. Eliminar filas con Quantity <= 0 (devoluciones y errores) ---
df = df[df["Quantity"] > 0]
   print(f"[LIMPIEZA] Tras filtrar Quantity > 0: {len(df)} filas.")

   # --- 3. Eliminar filas con UnitPrice <= 0 (registros corruptos) ---
df = df[df["UnitPrice"] > 0]
   print(f"[LIMPIEZA] Tras filtrar UnitPrice > 0: {len(df)} filas.")

# --- 4. Eliminar duplicados exactos ---
antes_dup = len(df)
df = df.drop_duplicates()
   print(f"[LIMPIEZA] Duplicados eliminados: {antes_dup - len(df)}.")

   # --- 5. Gestión de nulos ---

   # Description: rellenar nulos con 'Sin descripción'
df["Description"] = df["Description"].fillna("Sin descripción")

   # CustomerID: rellenar nulos con 0 (cliente anónimo / sin registro)
   # Se convierte a int después de rellenar para evitar decimales
df["CustomerID"] = df["CustomerID"].fillna(0).astype(int)

   print(f"[LIMPIEZA] Nulos gestionados en Description y CustomerID.")

# --- 6. Normalización de fechas a formato YYYY-MM-DD ---
   # InvoiceDate ya viene como datetime, extraemos solo la parte de fecha
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"]).dt.date
   print(f"[LIMPIEZA] Fechas normalizadas a formato DATE.")

   # --- 7. Normalización de texto ---

   # Description: strip de espacios, capitalización consistente
df["Description"] = (
    df["Description"]
   .astype(str)
   .str.strip()
   .str.upper()
   )

   # StockCode: convertir a string, eliminar espacios
df["StockCode"] = df["StockCode"].astype(str).str.strip().str.upper()

   print(f"[LIMPIEZA] Texto normalizado en Description y StockCode.")

   # --- 8. Calcular total_venta ---
df["total_venta"] = (df["Quantity"] * df["UnitPrice"]).round(2)

   # --- 9. Renombrar columnas al esquema interno ---
df = df.rename(columns={
```

```
        "StockCode": "id_producto",
        "Description":  "nombre",
        "InvoiceDate":  "fecha_venta",
        "Quantity":  "unidades_vendidas",
        "UnitPrice": "precio_unitario",
        "InvoiceNo": "id_venta_original",
        })

        print(f"[LIMPIEZA] Pipeline completado. Filas finales: {len(df)} (de
{filas_inicio} originales).")
        print(f"[LIMPIEZA] Filas eliminadas en total: {filas_inicio - len(df)}.")

        return df


def split_productos_ventas(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
        """
    Separa el DataFrame limpio en las dos tablas del modelo de datos:
        - Productos (catálogo único de productos)
        - Ventas (registros de transacciones)

    Parámetros:
         df (pd.DataFrame): DataFrame limpio devuelto por clean_and_normalize().

    Retorna:
        tuple: (df_productos, df_ventas)
        """

        # --- Tabla Productos ---
        # Un producto único se identifica por id_producto (StockCode)
        # Se mantiene el precio unitario más reciente si hay variación
    df_productos = (
         df[["id_producto", "nombre", "precio_unitario"]]
        .sort_values("precio_unitario", ascending=False)
        .drop_duplicates(subset=["id_producto"], keep="first")
        .reset_index(drop=True)
        )

        # Añadir categoría vacía (el dataset no tiene categoría; se puede
enriquecer)
    df_productos["categoria"] = "Sin categoría"

        # Reordenar columnas según el modelo de BD
    df_productos = df_productos[["id_producto", "nombre", "categoria",
"precio_unitario"]]

        print(f"[SPLIT] Productos únicos: {len(df_productos)}")

        # --- Tabla Ventas ---
        # Cada fila es una línea de transacción de venta
    df_ventas = df[[
        "id_venta_original",
        "id_producto",
        "fecha_venta",
        "unidades_vendidas",
        "total_venta",
        ]].copy().reset_index(drop=True)

        print(f"[SPLIT] Registros de ventas: {len(df_ventas)}")
```

```
        return df_productos, df_ventas
```

Archivo: load_db.py

Ruta completa: `src/pipeline/load_db.py`

```
"""
load_db.py — Capa de almacenamiento: inserción en Supabase (PostgreSQL)
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Responsabilidad:
    Recibe los DataFrames limpios (productos y ventas) y los inserta
    en la base de datos Supabase usando psycopg2 (PostgreSQL).

        También contiene la función de creación de tablas si no existen,
        lo que permite arrancar el sistema desde cero con una sola ejecución.

Configuración necesaria (archivo environment/.env en la raíz del proyecto):
        DB_HOST      = <host de Supabase, p.ej. aws-0-eu-west-1.pooler.supabase.com>
        DB_PORT      = 6543
        DB_NAME      = postgres
        DB_USER      = <usuario del proyecto Supabase>
        DB_PASSWORD = <contraseña del proyecto Supabase>

Nota sobre Supabase:
        Se eligió Supabase (PostgreSQL) frente a MySQL/XAMPP porque ofrece
    una instancia gratuita accesible remotamente, lo que facilita la
    integración con el dashboard y el modelo de predicción sin depender
        de que un servidor local esté levantado.
"""

import pandas as pd
from psycopg2.extras import execute_values


def create_tables(conn) -> None:
        """
    Recrea desde cero las tres tablas del modelo de datos.

        Primero hace DROP TABLE IF EXISTS ... CASCADE para eliminar versiones
    antiguas del esquema (por ejemplo si una ejecución previa creó
    id_producto como INTEGER en lugar de VARCHAR) y a continuación las
    crea con el esquema correcto. De esta forma el esquema del código
        es siempre la única fuente de verdad.

        Como el pipeline ya reescribe todo el dataset en cada ejecución,
    borrar las tablas no supone pérdida de información real.

    Tablas creadas:
        - productos
        - ventas
        - predicciones (vacía, para uso del módulo de ML)
```

```
    Parámetros:
      conn: Conexión activa de psycopg2.
      """

    sql_create = """
      -- Se eliminan antes de crear para garantizar el esquema correcto
      DROP TABLE IF EXISTS predicciones, ventas, productos CASCADE;

      -- Tabla de productos (catálogo)
      CREATE TABLE productos (
           id_producto     VARCHAR(20)        PRIMARY KEY,
           nombre          VARCHAR(255)       NOT NULL,
           categoria       VARCHAR(100)       DEFAULT 'Sin categoría',
           precio_unitario DECIMAL(10, 2)  NOT NULL
       );

      -- Tabla de ventas (transacciones históricas)
      CREATE TABLE ventas (
           id_venta             SERIAL       PRIMARY KEY,
           id_venta_original   VARCHAR(20),
           id_producto         VARCHAR(20)    REFERENCES productos(id_producto),
           fecha_venta          DATE         NOT NULL,
           unidades_vendidas   INT            NOT NULL CHECK (unidades_vendidas >
0),
           total_venta         DECIMAL(10, 2)  NOT NULL
       );

      -- Tabla de predicciones (la rellena el módulo ML de Deninson)
      CREATE TABLE predicciones (
           id_prediccion        SERIAL       PRIMARY KEY,
           id_producto         VARCHAR(20)    REFERENCES productos(id_producto),
           fecha_prediccion     DATE         NOT NULL,
           unidades_predichas  INT,
           unidades_vendidas   INT,
           confianza_modelo    DECIMAL(5, 4)
       );
      """

      with conn.cursor() as cur:
       cur.execute(sql_create)
    conn.commit()
      print("[DB] Tablas recreadas desde cero: productos, ventas, predicciones.")


def insert_productos(conn, df_productos: pd.DataFrame) -> None:
      """
    Inserta el catálogo de productos en la tabla 'productos'.
      Usa ON CONFLICT DO NOTHING para evitar errores por duplicados
    si el pipeline se ejecuta varias veces.

    Parámetros:
      conn: Conexión activa de psycopg2.
       df_productos (pd.DataFrame): DataFrame con columnas:
           id_producto, nombre, categoria, precio_unitario
      """

      # Convertir a lista de tuplas para la inserción en lote
      records = list(df_productos.itertuples(index=False, name=None))
```

```
    sql = """
       INSERT INTO productos (id_producto, nombre, categoria, precio_unitario)
       VALUES %s
       ON CONFLICT (id_producto) DO NOTHING;
       """

       with conn.cursor() as cur:
        execute_values(cur, sql, records)
    conn.commit()

       print(f"[DB] Productos insertados: {len(records)} registros.")


def insert_ventas(conn, df_ventas: pd.DataFrame, batch_size: int = 5000) -> None:
       """
    Inserta los registros de ventas en la tabla 'ventas' por lotes.
       Se inserta por lotes para no saturar la memoria ni la conexión
       con un dataset de más de 500.000 filas.

    Parámetros:
       conn: Conexión activa de psycopg2.
        df_ventas (pd.DataFrame): DataFrame con columnas:
            id_venta_original, id_producto, fecha_venta,
            unidades_vendidas, total_venta
        batch_size (int): Número de filas por lote. Por defecto 5.000.
       """

       total = len(df_ventas)
    insertados = 0

    sql = """
       INSERT INTO ventas
              (id_venta_original, id_producto, fecha_venta, unidades_vendidas,
total_venta)
       VALUES %s;
       """

       with conn.cursor() as cur:
       # Iterar en lotes para no bloquear memoria
       for inicio in range(0, total, batch_size):
            lote = df_ventas.iloc[inicio: inicio + batch_size]
             records = list(lote.itertuples(index=False, name=None))
            execute_values(cur, sql, records)
            insertados += len(records)
             print(f"[DB] Ventas insertadas: {insertados}/{total}...")

    conn.commit()
       print(f"[DB] Total ventas insertadas: {insertados} registros.")


def insert_predicciones(conn, df_pred: pd.DataFrame, batch_size: int = 5000) ->
None:
       """
    Inserta las predicciones per-producto en la tabla 'predicciones'.

       El modelo se entrena sobre la serie agregada diaria por robustez
    estadística (la mayoría de SKUs tienen histórico escaso). El
    orquestador run_pipeline.py descompone las predicciones globales
       per-producto aplicando la cuota histórica de ventas de cada SKU y
```

```
    pasa aquí el DataFrame ya explotado.

    Parámetros:
       conn: Conexión activa de psycopg2.
        df_pred (pd.DataFrame): Columnas requeridas:
            id_producto, fecha, unidades_predichas, unidades_vendidas, confianza
        batch_size (int): Filas por lote. Necesario porque el cross-join
              de SKUs × días predichos genera cientos de miles de filas.
       """
       if df_pred is None or df_pred.empty:
       print("[DB] No hay predicciones que insertar.")
       return

       records = [
       (
            row.id_producto,
            row.fecha,
             int(round(row.unidades_predichas)),
             int(round(row.unidades_vendidas)),
             float(row.confianza) if row.confianza is not None else None,
       )
       for row in df_pred.itertuples(index=False)
       ]

    sql = """
       INSERT INTO predicciones
              (id_producto, fecha_prediccion, unidades_predichas,
             unidades_vendidas, confianza_modelo)
       VALUES %s;
       """

       total = len(records)
    insertados = 0
       with conn.cursor() as cur:
       for inicio in range(0, total, batch_size):
            lote = records[inicio: inicio + batch_size]
            execute_values(cur, sql, lote)
            insertados += len(lote)
             print(f"[DB] Predicciones insertadas: {insertados}/{total}...")
    conn.commit()
       print(f"[DB] Total predicciones insertadas: {total} registros.")


def close_connection(conn) -> None:
       """
       Cierra la conexión a la base de datos de forma limpia.

    Parámetros:
       conn: Conexión activa de psycopg2.
       """
       if conn and not conn.closed:
        conn.close()
       print("[DB] Conexión cerrada correctamente.")
```

Archivo: run_pipeline.py

Ruta completa: `src/pipeline/run_pipeline.py`

```python
"""
run_pipeline.py — Orquestador del pipeline de datos
Proyecto: StockPulse
Autor: Sergio Romero (Data Engineer)

Responsabilidad:
       Punto de entrada único del pipeline end-to-end.
    Ejecuta en orden:
       1. Ingesta                → ingest.py
       2. Limpieza               → clean.py
       3. Carga de datos BD      → load_db.py (productos + ventas)
       4. Entrenamiento ML       → models/train_model.py (modelo agregado)
       5. Descomposición per-SKU → cuota histórica sobre predicción global
       6. Carga de predicciones → load_db.py (tabla predicciones per-producto)

       También guarda una copia del dataset limpio en data/processed/
    que consume el módulo de ML.

Uso:
       python src/pipeline/run_pipeline.py --input data/raw/online_retail.xlsx
       python src/pipeline/run_pipeline.py --input data/raw/online_retail.csv
"""

import argparse
import os
import sys

# Añadimos la raíz del proyecto al path para poder hacer imports con "src.*"
# desde cualquier sitio (útil cuando lo ejecutáis desde la raíz del repo).
sys.path.append(os.path.join(os.path.dirname(__file__), "..", ".."))

# get_connection vive en src/backend/database.py y lo reutilizamos aquí.
from src.backend.database import get_connection
from src.pipeline.ingest import load_dataset
from src.pipeline.clean import clean_and_normalize, split_productos_ventas
from src.pipeline.load_db import (
    create_tables,
    insert_productos,
    insert_ventas,
    insert_predicciones,
    close_connection,
)
from src.models.train_model import run_training


# Ruta donde se guarda el dataset procesado para el módulo ML
PROCESSED_PATH = os.path.join("data", "processed", "ventas_limpias.csv")


def run_pipeline(filepath: str) -> None:
       """
    Ejecuta el pipeline completo de datos.

       Pasos:
       1. Carga el dataset desde la ruta indicada.
       2. Aplica limpieza y normalización.
       3. Separa en tablas Productos y Ventas.
```

```
   4. Guarda el dataset limpio como CSV para el módulo ML.
   5. Conecta a Supabase, crea tablas e inserta productos y ventas.
   6. Entrena el modelo Ridge sobre el CSV limpio.
   7. Inserta las predicciones en la tabla 'predicciones' para el dashboard.

Parámetros:
    filepath (str): Ruta al archivo CSV o XLSX de entrada.
   """

print("\n" + "=" * 60)
print("   STOCKPULSE — Pipeline de datos")
print("=" * 60 + "\n")

   # ---- PASO 1: INGESTA ----
print(">>> PASO 1: Ingesta de datos")
df_raw = load_dataset(filepath)

   # ---- PASO 2: LIMPIEZA ----
   print("\n>>> PASO 2: Limpieza y normalización")
df_clean = clean_and_normalize(df_raw)

   # ---- PASO 3: SEPARACIÓN EN TABLAS ----
   print("\n>>> PASO 3: Separación en tablas Productos / Ventas")
df_productos, df_ventas = split_productos_ventas(df_clean)

   # ---- PASO 4: GUARDAR CSV LIMPIO PARA ML ----
   print("\n>>> PASO 4: Guardando dataset limpio para módulo ML")
os.makedirs(os.path.dirname(PROCESSED_PATH), exist_ok=True)
df_clean.to_csv(PROCESSED_PATH, index=False, encoding="utf-8")
   print(f"[PIPELINE] Dataset limpio guardado en: {PROCESSED_PATH}")

   # ---- PASO 5: INSERCIÓN EN BASE DE DATOS ----
   print("\n>>> PASO 5: Inserción en Supabase (productos + ventas)")
   conn = None
   try:
   conn = get_connection()
   # create_tables hace DROP + CREATE, así cada ejecución refleja
   # exactamente el CSV de entrada y garantiza el esquema correcto.
    create_tables(conn)
    insert_productos(conn, df_productos)
    insert_ventas(conn, df_ventas)

   # ---- PASO 6: ENTRENAMIENTO DEL MODELO ----
   # Encadenamos el train_model aquí: entrena Ridge y nos devuelve el
   # DataFrame de predicciones globales (columnas: fecha, ventas_reales,
   # ventas_predichas) listo para descomponer per-producto.
   print("\n>>> PASO 6: Entrenamiento del modelo (Ridge)")
    df_pred = run_training(PROCESSED_PATH)

   # ---- PASO 7: DESCOMPOSICIÓN PER-PRODUCTO ----
   # El modelo predice demanda diaria agregada; el dashboard la espera
   # por SKU. Repartimos cada predicción global entre los productos
   # según su cuota histórica de ventas, de forma que la tabla
   # predicciones tenga una fila por (id_producto, fecha).
   print("\n>>> PASO 7: Descomposición per-producto por cuota histórica")
   from sklearn.metrics import r2_score

    cuotas = df_ventas.groupby("id_producto")["unidades_vendidas"].sum()
    cuotas = (cuotas / cuotas.sum()).rename("cuota").reset_index()
```

```
        print(f"[PIPELINE] Cuotas calculadas para {len(cuotas)} productos.")

        # R² del modelo como "confianza" para el KPI del dashboard.
        # Se clipa a [0,1] porque el frontend lo pinta como porcentaje.
        r2 = float(r2_score(df_pred["ventas_reales"], df_pred["ventas_predichas"]))
        confianza = max(0.0, min(1.0, r2))
        print(f"[PIPELINE] R² del modelo = {r2:.4f} → confianza = {confianza:.4f}")

         df_pred_exp = df_pred.merge(cuotas, how="cross")
         df_pred_exp["unidades_predichas"] = (
             df_pred_exp["ventas_predichas"] * df_pred_exp["cuota"]
         ).round()
         df_pred_exp["unidades_vendidas"] = (
             df_pred_exp["ventas_reales"] * df_pred_exp["cuota"]
         ).round()
         df_pred_exp["confianza"] = confianza
         df_pred_exp = df_pred_exp[
              ["id_producto", "fecha", "unidades_predichas",
              "unidades_vendidas", "confianza"]
        ]
        print(f"[PIPELINE] Filas de predicciones a insertar: {len(df_pred_exp)}")

        # ---- PASO 8: INSERCIÓN DE PREDICCIONES EN BD ----
        # Push final: el dashboard leerá esta tabla para pintar las gráficas.
        print("\n>>> PASO 8: Inserción de predicciones en Supabase")
         insert_predicciones(conn, df_pred_exp)
        except Exception as e:
        print(f"\n[ERROR] Fallo durante la fase BD/entrenamiento: {e}")
        print("[INFO] Los datos limpios sí se guardaron en data/processed/")
        raise
        finally:
         close_connection(conn)

    print("\n" + "=" * 60)
    print("   Pipeline completado con éxito.")
    print("=" * 60 + "\n")


# ---- Punto de entrada por línea de comandos ----
if __name__ == "__main__":
        parser = argparse.ArgumentParser(
        description="StockPulse — Pipeline de ingesta, limpieza y carga de datos."
        )
    parser.add_argument(
        "--input",
        type=str,
        required=True,
        help="Ruta al archivo de datos (CSV o XLSX). Ej:
data/raw/online_retail.xlsx",
        )
    args = parser.parse_args()
    run_pipeline(args.input)
```

7. Breve explicación por módulo

src/pipeline/ — Data Engineer (Sergio Romero)

Módulo responsable de todo el proceso ETL desde la carga del dataset hasta la inserción en Supabase.

- **ingest.py**: carga el archivo CSV o XLSX, valida que las columnas obligatorias estén presentes y devuelve un DataFrame en bruto listo para limpiar.
- **clean.py**: aplica 9 operaciones de limpieza y normalización: eliminación de cancelaciones, filtrado de cantidades y precios negativos, deduplicación, gestión de nulos, normalización de fechas y texto, cálculo de total_venta y separación en tablas Productos y Ventas.
- **load_db.py**: gestiona la conexión a Supabase mediante psycopg2, crea las tablas si no existen e inserta los datos en lotes de 5.000 filas para optimizar el rendimiento con datasets grandes.
- **run_pipeline.py**: orquestador del pipeline. Punto de entrada único que encadena ingest → clean → load_db y exporta el dataset limpio como CSV para el módulo de ML.
- **__init__.py**: inicialización del módulo de pipeline. Permite importar las funciones de ingesta, limpieza e inserción desde otros módulos del proyecto.

src/models/ — Data Scientist (Deninson Tapia)

Módulo responsable del entrenamiento del modelo de predicción de demanda y la generación de predicciones futuras.

- **features.py**: preprocesamiento previo al entrenamiento. Aplica tratamiento de outliers con IQR, transformación logarítmica para reducir el sesgo de la distribución de ventas, y genera 21 features temporales: lags de 1, 3, 7, 14, 21 y 30 días; medias móviles y desviación estándar móvil en ventanas de 3, 7, 14 y 30 días; y variables de calendario como día de la semana, mes, trimestre y si es fin de semana.
- **train_model.py**: script de entrenamiento del modelo final Ridge Regression con alpha=10.0. Carga el CSV limpio generado por el pipeline, aplica el preprocesamiento completo, escala las features con StandardScaler, entrena el modelo, evalúa sobre el conjunto de test obteniendo RMSE 5069.20, MAE 4355.06 y R^2 0.6262, y exporta el modelo serializado (ridge_model.pkl), el scaler (scaler.pkl) y las predicciones (predicciones_ridge.csv).

src/frontend/ — Business Intelligence (Jimmy Chen)

Módulo responsable de la interfaz de usuario web del sistema **StockPulse**. Desarrollado en HTML, CSS y JavaScript puro sin dependencias de frameworks, con Chart.js para la visualización de gráficos.

- **index.html**: estructura completa de la aplicación. Incluye navbar con gestión dinámica de estados de sesión, página de inicio con área de carga de archivos CSV y XLSX, modal de login, páginas estáticas de Quiénes somos y Contacto, y dashboard con cuatro KPIs y cuatro gráficos interactivos con selector de producto global.
- **styles.css**: sistema de diseño completo de la aplicación. Define variables CSS para la paleta de colores (navy, verde, blanco), componentes reutilizables como navbar, botones, tarjetas KPI, modal y gráficos, y diseño responsive para dispositivos móviles.

- **app.js**: lógica completa del frontend. Gestiona la navegación entre páginas sin recarga, el sistema de autenticación con sesión almacenada exclusivamente en memoria sin persistencia en disco, la validación y envío de archivos al backend con control de errores por formato y tamaño, la carga del dashboard con KPIs y productos, y el renderizado de los cuatro gráficos conectados a los endpoints REST del backend.

src/backend/ — Platform / Project Manager (Joaquín Fernández)

Módulo responsable de la API REST que conecta el frontend con los datos de Supabase y el modelo de predicción.

- **main.py**: punto de entrada de la aplicación FastAPI. Registra todos los routers de la carpeta routes/ y arranca el servidor.
- **auth.py**: lógica de autenticación. Gestiona la verificación de credenciales, la generación de tokens de sesión y la validación de acceso a los endpoints protegidos.
- **database.py**: gestiona la conexión a Supabase y expone las funciones de consulta que utilizan el resto de módulos del backend.
- **routes/**: carpeta que agrupa los endpoints de la API organizados por funcionalidad.
    - **__init__.py**: inicialización del módulo de rutas.
    - **auth_routes.py**: endpoints de autenticación - login y logout.
    - **dashboard_routes.py**: endpoints que sirven los datos del dashboard - KPIs, productos y datos de los cuatro gráficos.
    - **upload_routes.py**: endpoint de carga de archivos CSV y XLSX. Recibe el archivo del frontend, lanza el pipeline de limpieza y devuelve confirmación al cliente.

environment/

Carpeta de configuración del entorno de desarrollo.

- **requirements.txt**: lista de todas las dependencias del proyecto con sus versiones exactas. Permite reproducir el entorno con un único comando pip install -r environment/requirements.txt.

docs/

Carpeta de documentación del proyecto.

- **Fase0_Presentacion_alineacion_viabilidad_Grupo5.pdf**: documento de presentación inicial del proyecto.
- **Fase1_Grupo5.pdf**: documento de diseño y planificación de la Fase 1.
- **Fase2_Producto_Funcional_Local_y_Codigo_Grupo5.pdf**: este documento, correspondiente a la Fase 2.

8. Limitaciones actuales

A pesar de los avances realizados, el sistema presenta todavía varias limitaciones que deben ser abordadas en futuras fases del proyecto.

8.1 Bugs detectados

- Cierre de sesión, sale al inicio en dos clics no en uno.
- Errores de conexión con la base de datos en caso de que las variables de entorno no estén bien configuradas.

8.2 Atajos implementados

- Validación de datos simplificada priorizando funcionalidad sobre seguridad y robustez.
- Ejecución automática del pipeline, mediante la ejecución a través de un botón en el dashboard.

8.3 Partes incompletas

- Frontend en estado inicial, interfaz funcional pero con posibilidad de mejoras de accesibilidad y formularios de configuración.
- Falta de test automatizados.